Object Oriented Programming (OOP) - Practical Submission

JAVA PRACTICAL FILE

INDEX

COURSE CODE CEUE203 – OOP **STUDENT ID 25IT048** **STUDENT NAME Harsh Kunpara**

Sr. No.	Practical	Submission Date
1	PRACTICAL 1	07/07/2026
2	PRACTICAL 2	13/07/2026
3	PRACTICAL 3	20/07/2026
4	PRACTICAL 4	29/07/2026
5	PRACTICAL 5	03/08/2026
6	PRACTICAL 6	17/08/2026
7	PRACTICAL 7	27/10/2026
8	PRACTICAL 8	16/09/2026
9	PRACTICAL 9	16/09/2026
10	PRACTICAL 10	05/10/2026
11	PRACTICAL 11	27/10/2026
12	PRACTICAL 12	27/10/2026
13	PRACTICAL 13	27/10/2026
14	PRACTICAL 14	27/10/2026

PRACTICAL – 1

Problem 1: Vending Machine

Aim: To simulate a vending machine using enum, Scanner, switch expression, and a running total of inserted coins.

Code:

```
import java.util.Scanner;

public class VendingMachine {
   enum Coin {
      ONE, TWO, FIVE, TEN
   }

   public static void main(String[] args) {
      final int PRICE = 15;
      int total = 0;

      Scanner sc = new Scanner(System.in);

      while (total < PRICE) {
         System.out.print("Enter coin (ONE, TWO, FIVE, TEN): ");
         Coin coin = Coin.valueOf(sc.next().toUpperCase());

         int value = switch (coin) {
            case ONE -> 1;
            case TWO -> 2;
            case FIVE -> 5;
            case TEN -> 10;
         };

         total += value;
         System.out.println("Total so far: " + total);
      }

      System.out.println("Paid. Change: " + (total - PRICE));
      sc.close();
   }
}
```

Output:

```
Enter coin (ONE, TWO, FIVE, TEN): TEN
Total so far: 10
Enter coin (ONE, TWO, FIVE, TEN): FIVE
Total so far: 15
Paid. Change: 0
```

Learning Outcome: Learned to use enum constants and switch expressions to process coin values, update a running total, and calculate the change.

Problem 2: Toll Booth

Aim: To calculate toll collection and count vehicle types using a record, switch expression, and counters.

Code:

```
import java.util.Scanner;

public class TollBooth {
   record Vehicle(String number, String type) { }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);

      int totalToll = 0;
      int bike = 0, car = 0, truck = 0;

      while (true) {
         System.out.print("Enter vehicle number (or done): ");
         String number = sc.nextLine();

         if (number.equalsIgnoreCase("done")) {
            break;
         }

         System.out.print("Enter vehicle type (bike/car/truck): ");
         String type = sc.nextLine().toLowerCase();

         Vehicle vehicle = new Vehicle(number, type);

         int toll = switch (vehicle.type()) {
            case "bike" -> 20;
            case "car" -> 50;
            case "truck" -> 150;
            default -> 0;
         };

         if (toll == 0) {
            System.out.println("Invalid vehicle type.");
            continue;
         }

         totalToll += toll;

         switch (vehicle.type()) {
            case "bike" -> bike++;
            case "car" -> car++;
            case "truck" -> truck++;
         }
      }

      System.out.println("Total toll: " + totalToll);

      String mostFrequent = "bike";
      int highest = bike;

      if (car > highest) {
         mostFrequent = "car";
         highest = car;
      }
      if (truck > highest) {
         mostFrequent = "truck";
      }
```

```
        System.out.println("Most frequent: " + mostFrequent);
        sc.close();
    }
}
```

Output:

```
Enter vehicle number (or done): GJ13AB1234
Enter vehicle type (bike/car/truck): car
Enter vehicle number (or done): GJ13CD5678
Enter vehicle type (bike/car/truck): car
Enter vehicle number (or done): GJ13EF9012
Enter vehicle type (bike/car/truck): bike
Enter vehicle number (or done): done
Total toll: 120
Most frequent: car
```

Learning Outcome: Learned to represent vehicle data using a record, calculate tolls with a switch expression, and maintain counts for different vehicle types.

Problem 3: Rock-Paper-Scissors-Lizard-Spock (Advanced)

Aim: To implement the RPSLS game using enum, switch expressions, random selection, and score tracking.

Code:

```
import java.util.Random;
import java.util.Scanner;

public class RPSLS {
   enum Move {
      ROCK, PAPER, SCISSORS, LIZARD, SPOCK
   }

   static int winner(Move a, Move b) {
      if (a == b) {
         return 0;
      }

      boolean aWins = switch (a) {
         case ROCK -> b == Move.SCISSORS || b == Move.LIZARD;
         case PAPER -> b == Move.ROCK || b == Move.SPOCK;
         case SCISSORS -> b == Move.PAPER || b == Move.LIZARD;
         case LIZARD -> b == Move.SPOCK || b == Move.PAPER;
         case SPOCK -> b == Move.SCISSORS || b == Move.ROCK;
      };

      return aWins ? 1 : -1;
   }

   public static void main(String[] args) {
      Scanner sc = new Scanner(System.in);
      Random random = new Random();
      Move[] moves = Move.values();

      int playerScore = 0;
      int computerScore = 0;

      for (int round = 1; round <= 5; round++) {
         System.out.print("Round " + round
               + " - Enter ROCK, PAPER, SCISSORS, LIZARD, SPOCK: ");
         Move player = Move.valueOf(sc.next().toUpperCase());
         Move computer = moves[random.nextInt(moves.length)];

         int result = winner(player, computer);
         System.out.println("You: " + player + " | Computer: " + computer);

         if (result == 1) {
            playerScore++;
            System.out.println("You win this round.");
         } else if (result == -1) {
            computerScore++;
            System.out.println("Computer wins this round.");
         } else {
            System.out.println("This round is a tie.");
         }
      }

      System.out.println("Final score: You " + playerScore
            + " - " + computerScore + " Computer");

      if (playerScore > computerScore) {
         System.out.println("You win!");
      } else if (computerScore > playerScore) {
```

```
            System.out.println("Computer wins!");
        } else {
            System.out.println("Overall tie!");
        }

        sc.close();
    }
}
```

Output:

```
Round 1 - Enter ROCK, PAPER, SCISSORS, LIZARD, SPOCK: ROCK
You: ROCK | Computer: SCISSORS
You win this round.
Round 2 - Enter ROCK, PAPER, SCISSORS, LIZARD, SPOCK: PAPER
You: PAPER | Computer: ROCK
You win this round.
Round 3 - Enter ROCK, PAPER, SCISSORS, LIZARD, SPOCK: SPOCK
You: SPOCK | Computer: LIZARD
Computer wins this round.
Round 4 - Enter ROCK, PAPER, SCISSORS, LIZARD, SPOCK: LIZARD
You: LIZARD | Computer: SPOCK
You win this round.
Round 5 - Enter ROCK, PAPER, SCISSORS, LIZARD, SPOCK: SCISSORS
You: SCISSORS | Computer: ROCK
Computer wins this round.
Final score: You 3 - 2 Computer
You win!
```

Learning Outcome: Learned to define game rules with enum and switch expressions, generate random computer moves, compare moves, and maintain scores across rounds.

PRACTICAL – 2

Problem 1: Smart Thermostat

Aim: To model a smart thermostat using encapsulation, constructors, constructor chaining, static members, and controlled temperature operations.

Code:

```
public class Thermostat {
    private String location;
    private int temperature;

    private static final int MIN = 16;
    private static final int MAX = 30;
    private static int activeCount = 0;

    public Thermostat(String location, int startTemp) {
        this.location = location;

        if (startTemp >= MIN && startTemp <= MAX) {
            this.temperature = startTemp;
        } else {
            this.temperature = 22;
        }

        activeCount++;
    }

    public Thermostat(String location) {
        this(location, 22);
    }

    public void raise() {
        if (temperature < MAX) {
            temperature++;
        } else {
            System.out.println("Already at maximum (30)");
        }
    }

    public void lower() {
        if (temperature > MIN) {
            temperature--;
        } else {
            System.out.println("Already at minimum (16)");
        }
    }

    public int getTemperature() {
        return temperature;
    }

    public static int getActiveCount() {
        return activeCount;
    }

    public static void main(String[] args) {
        Thermostat room1 = new Thermostat("Living Room", 25);
        Thermostat room2 = new Thermostat("Bedroom");
```

```
        for (int i = 0; i < 10; i++) {
            room1.raise();
            System.out.println("Temperature: " + room1.getTemperature());
        }

        for (int i = 0; i < 20; i++) {
            room1.lower();
            System.out.println("Temperature: " + room1.getTemperature());
        }

        System.out.println("Active thermostats: " + Thermostat.getActiveCount());
    }
}
```

Output:

```
Temperature: 26
Temperature: 27
Temperature: 28
Temperature: 29
Temperature: 30
Already at maximum (30)
Temperature: 30
Already at maximum (30)
Temperature: 30
Already at maximum (30)
Temperature: 30
Already at maximum (30)
Temperature: 30
Already at maximum (30)
Temperature: 30
Temperature: 29
Temperature: 28
Temperature: 27
Temperature: 26
Temperature: 25
Temperature: 24
Temperature: 23
Temperature: 22
Temperature: 21
Temperature: 20
Temperature: 19
Temperature: 18
Temperature: 17
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
Temperature: 16
Already at minimum (16)
```

```
Temperature: 16
Active thermostats: 2
```

Learning Outcome: Learned to apply encapsulation, constructor overloading and chaining, static variables and methods, and boundary checks for temperature control.

Problem 2: Cinema Show

Aim: To manage cinema seat bookings and cancellations using constructors, final fields, static booking totals, and validation.

Code:

```
public class CinemaShow {
   private String title;
   private int seatsAvailable;
   private final int capacity;
   private static int totalBooked = 0;

   public CinemaShow(String title, int capacity) {
      this.title = title;
      this.capacity = capacity;
      this.seatsAvailable = capacity;
   }

   public CinemaShow(String title) {
      this(title, 100);
   }

   public boolean book(int n) {
      if (n > 0 && n <= seatsAvailable) {
         seatsAvailable -= n;
         totalBooked += n;
         return true;
      }
      return false;
   }

   public void cancel(int n) {
      if (n > 0) {
         seatsAvailable = Math.min(capacity, seatsAvailable + n);
      }
   }

   public int getSeatsAvailable() {
      return seatsAvailable;
   }

   public static int getTotalBooked() {
      return totalBooked;
   }

   public static void main(String[] args) {
      CinemaShow show = new CinemaShow("Java: The Movie", 50);

      System.out.println("Book 20 seats: " + show.book(20));
      System.out.println("Seats available: " + show.getSeatsAvailable());

      System.out.println("Book 35 seats: " + show.book(35));
      System.out.println("Seats available: " + show.getSeatsAvailable());

      System.out.println("Book 15 seats: " + show.book(15));
      System.out.println("Seats available: " + show.getSeatsAvailable());

      show.cancel(5);
      System.out.println("After cancelling 5 seats: "
            + show.getSeatsAvailable());

      System.out.println("Total booked: " + CinemaShow.getTotalBooked());
```

```
  }
}
```

Output:

```
Book 20 seats: true
Seats available: 30
Book 35 seats: false
Seats available: 30
Book 15 seats: true
Seats available: 15
After cancelling 5 seats: 20
Total booked: 35
```

Learning Outcome: Learned to use constructors, final and static fields, validate booking requests, update available seats, and maintain the total number of successfully booked seats.

Problem 3: Parking Lot (Advanced)

Aim: To simulate parking management with capacity limits, separate vehicle counts, static revenue tracking, and park/leave operations.

Code:

```
public class ParkingLot {
   private int twoWheelers;
   private int fourWheelers;

   private final int twoCap;
   private final int fourCap;

   private static long revenue = 0;

   public ParkingLot(int twoCap, int fourCap) {
      this.twoCap = twoCap;
      this.fourCap = fourCap;
      this.twoWheelers = 0;
      this.fourWheelers = 0;
   }

   public void park(String type) {
      if (type.equalsIgnoreCase("two")) {
         if (twoWheelers < twoCap) {
            twoWheelers++;
            revenue += 20;
            System.out.println("Two-wheeler parked.");
         } else {
            System.out.println("Full");
         }
      } else if (type.equalsIgnoreCase("four")) {
         if (fourWheelers < fourCap) {
            fourWheelers++;
            revenue += 40;
            System.out.println("Four-wheeler parked.");
         } else {
            System.out.println("Full");
         }
      } else {
         System.out.println("Invalid vehicle type.");
      }
   }

   public void leave(String type) {
      if (type.equalsIgnoreCase("two")) {
         if (twoWheelers > 0) {
            twoWheelers--;
            System.out.println("Two-wheeler left.");
         } else {
            System.out.println("No two-wheeler to leave.");
         }
      } else if (type.equalsIgnoreCase("four")) {
         if (fourWheelers > 0) {
            fourWheelers--;
            System.out.println("Four-wheeler left.");
         } else {
            System.out.println("No four-wheeler to leave.");
         }
      }
   }
```

```
    public void displayStatus() {
        System.out.println("Two-wheelers: " + twoWheelers);
        System.out.println("Four-wheelers: " + fourWheelers);
        System.out.println("Revenue: " + revenue);
    }

    public static void main(String[] args) {
        ParkingLot lot = new ParkingLot(2, 1);

        lot.park("two");
        lot.park("two");
        lot.park("two");
        lot.park("four");
        lot.park("four");

        lot.leave("two");
        lot.park("two");

        System.out.println("Final occupancy:");
        lot.displayStatus();
    }
}
```

Output:

```
Two-wheeler parked.
Two-wheeler parked.
Full
Four-wheeler parked.
Full
Two-wheeler left.
Two-wheeler parked.
Final occupancy:
Two-wheelers: 2
Four-wheelers: 1
Revenue: 100
```

Learning Outcome: Learned to enforce parking capacity, maintain separate occupancy counts, process vehicle entry and exit, and calculate revenue using a static variable.

PRACTICAL -3

Aim: Distinct points (Point.java + Driver.java with main)

Code Screenshot:

Point.java

```java
import java.util.Objects;
class Point{
    private int x;
    private int y;

    public Point(int x,int y){
     this.x=x;
     this.y=y;
    }
    @Override
    public String toString() {

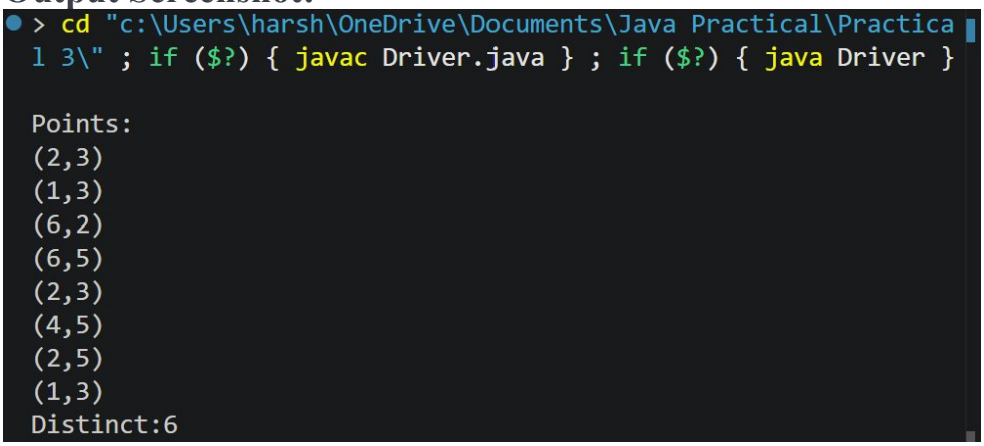
        return "("+x+","+y+")";
    }
    @Override
    public boolean equals(Object obj) {
        if(this==obj){
         return true;
        }
      if(obj==null||getClass()!=obj.getClass()){
        return false;
      }
      Point p=(Point)obj;
      return x==p.x&&y==p.y;
    }
    @Override
    public int hashCode() {
        return Objects.hash(x,y);
    }
}
```

Driver.java

```java
public class Driver {
    Run | Debug
    public static void main(String[] args) {
        Point[] points={
            new Point(x: 2,y: 3),
             new Point(x: 1,y: 3),
             new Point(x: 6,y: 2),
             new Point(x: 6,y: 5),
             new Point(x: 2,y: 3),
             new Point(x: 4,y: 5),
             new Point(x: 2,y: 5),
             new Point(x: 1,y: 3)
    };
    int count=0;
    for(int i=0;i<points.length;i++){
     boolean found=false;
     for(int j=0;j<i;j++){
     if(points[i].equals(points[j])){
 found=true;
 break;
     }
     }
     if(!found){
     count++;
     }
    }
    System.out.println(x: "Points:");
    for(Point p:points){
System.out.println(p);

    }
     System.out.println("Distinct:"+count);
}
}
```

Output Screenshot:



Learning Outcome:

Create a Java class with private data members and constructors.

Override the toString() method to provide a meaningful string representation of an object.

Override the equals() method to compare two objects based on their data instead of their memory addresses.

Override the hashCode() method while maintaining consistency with equals().

Compare objects stored in an array to identify unique (distinct) objects.

Apply object comparison techniques to count distinct objects without using Java Collections.

Aim: Duplicate card check (Card.java + driver.java)

Code Screenshot:



```java
import java.util.Objects;
public class Card {

    private String rank;
    private String suit;
    public Card(String rank,String suit){
        this.rank=rank;
        this.suit=suit;
    }
    @Override
    public String toString() {
        return rank+"of"+suit;
    }
    @Override
    public boolean equals(Object obj) {
        if(this==obj){
            return true;
        }
        if(obj==null||getClass()!=obj.getClass()){
        return false;
        }
        Card c= (Card) obj;
        return rank.equals(c.rank)&& suit.equals(c.suit);
    }
    @Override
    public int hashCode() {
        return Objects.hash(rank,suit);
    }
}
```

```java
public class diver {
    Run | Debug
    public static void main(String[] args) {
        Card[] cards=new Card[5];
        cards[0] = new Card(rank: "Ace", suit: "Spades");
        cards[1] = new Card(rank: "King", suit: "Hearts");
        cards[2] = new Card(rank: "Queen", suit: "Clubs");
        cards[3] = new Card(rank: "Ace", suit: "Spades");
        cards[4] = new Card(rank: "Jack", suit: "Diamonds");

        boolean Duplicatefound=false;
        for(int i=0;i<cards.length;i++){
            for(int j=0;j<i;j++){
             if(cards[i].equals(cards[j])){
              System.out.println("Duplicate:"+cards[i]);
              Duplicatefound=true;
              break;
             }
            }

        if(Duplicatefound){
            break;
        }
    }
}
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Documents\Java Practical\Practical 3
> cd "c:\Users\harsh\OneDrive\Documents\Java Practical\Practica
l 3\" ; if ($?) { javac diver.java } ; if ($?) { java diver }
Duplicate:AceofSpades
```

Learning Outcome:

Design a Java class representing real-world entities using encapsulation.

Implement custom object comparison by overriding equals() and hashCode().

Use the toString() method to display object information in a readable format.

Detect duplicate objects in an array using object equality.

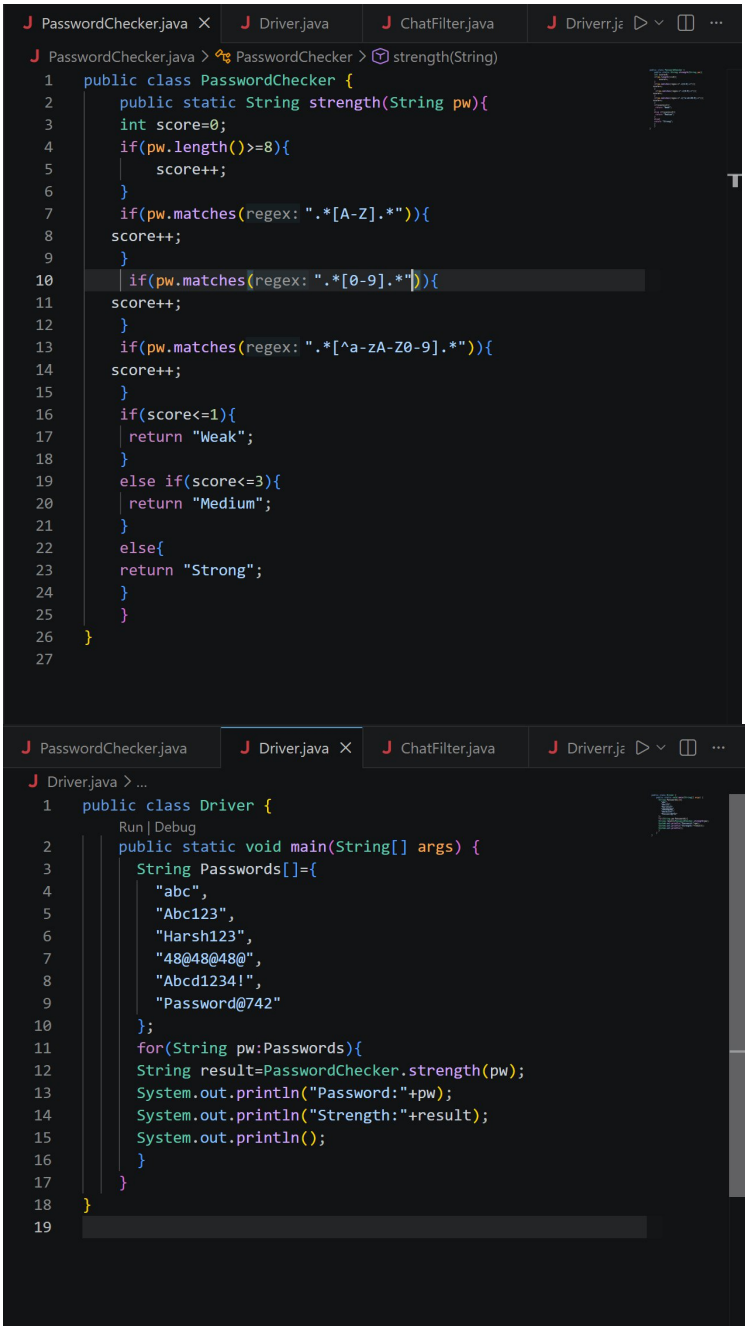
Understand the difference between reference equality (==) and logical equality (equals()).

Develop logic to identify duplicate entries before storing objects.

PRACTICAL -4

Aim: Password strength (PasswordChecker.java + Driver.java)

Code Screenshot:



```java
public class PasswordChecker {
    public static String strength(String pw){
    int score=0;
    if(pw.length()>=8){
        score++;
    }
    if(pw.matches(regex: ".*[A-Z].*")){
   score++;
    }
    if(pw.matches(regex: ".*[0-9].*")){
   score++;
    }
    if(pw.matches(regex: ".*[^a-zA-Z0-9].*")){
   score++;
    }
    if(score<=1){
     return "Weak";
    }
    else if(score<=3){
     return "Medium";
    }
    else{
    return "Strong";
    }
    }
}

```

```java
public class Driver {
    Run | Debug
    public static void main(String[] args) {
      String Passwords[]={
        "abc",
        "Abc123",
        "Harsh123",
        "48@48@48@",
        "Abcd1234!",
        "Password@742"
      };
      for(String pw:Passwords){
      String result=PasswordChecker.strength(pw);
      System.out.println("Password:"+pw);
      System.out.println("Strength:"+result);
      System.out.println();
      }
    }
}

```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Documents\Java Practical\Practical 4
> cd "c:\Users\harsh\OneDrive\Documents\Java Practical\Practica
l 4\" ; if ($?) { javac Driver.java } ; if ($?) { java Driver }

Password:abc
Strength:Weak

Password:Abc123
Strength:Medium

Password:Harsh123
Strength:Medium

Password:48@48@48@
Strength:Medium

Password:Abcd1234!
Strength:Strong

Password:Password@742
Strength:Strong

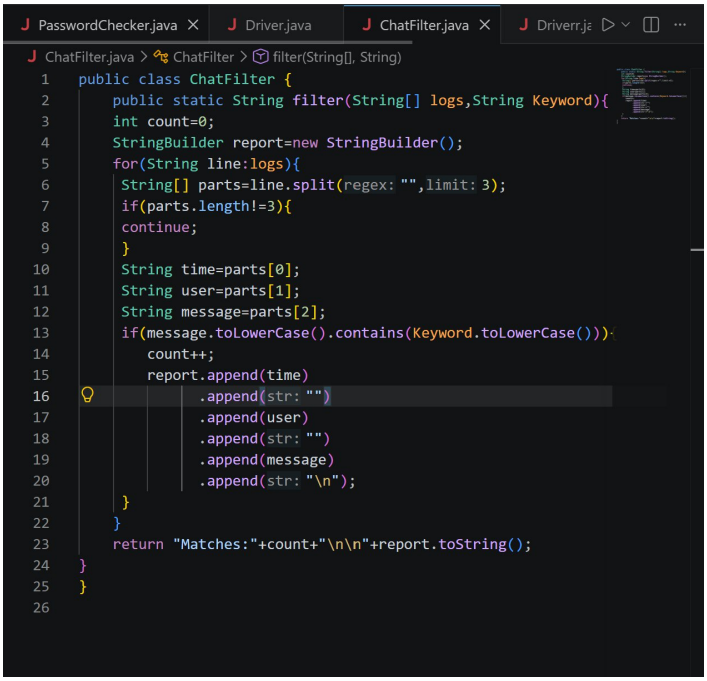
PS C:\Users\harsh\OneDrive\Documents\Java Practical\Practical 4
> 
```

Learning Outcome:

- Learned to validate passwords using multiple security rules.
- Understood the use of Regular Expressions (Regex) for pattern matching.
- Learned to use String methods such as length() and matches().
- Implemented password strength classification (Weak, Medium, Strong).
- Improved problem-solving and decision-making using conditional statements.

Aim: Chat-log filter (ChatFilter.java + Driver.java)

Code Screenshot:



```java
import java.util.Scanner;

public class Driverr {
    Run | Debug
    public static void main(String[] args) {
        String[] logs={
                "10:05 alice Hello there",
                "10:10 bob Good Morning",
                "WrongLine",
                "10:20 john Hello Harsh",
                "10:30 david Bye"
        };
        Scanner sc=new Scanner(System.in);
        System.out.println(x: "Enter Keyword:");
        String Keyword=sc.nextLine();
        String result=ChatFilter.filter(logs, Keyword);
        System.out.println(result);
        sc.close();
    }
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Documents\Java Practical\Practical 4
> cd "c:\Users\harsh\OneDrive\Documents\Java Practical\Practica
l 4\" ; if ($?) { javac Driverr.java } ; if ($?) { java Driverr
 }
Enter Keyword:
Hello
Matches:2

10:05 alice Hello there
10:20 john Hello Harsh

PS C:\Users\harsh\OneDrive\Documents\Java Practical\Practical 4
>
```

Learning Outcome:

- Learned to process and manipulate strings using split().
- Understood the use of StringBuilder for efficient string concatenation.
- Learned case-insensitive searching using toLowerCase() and contains().
- Implemented filtering of chat messages based on user input.

- Improved understanding of arrays, loops, and string processing techniques.

PRACTICAL -5

Aim: To implement abstraction, inheritance, and runtime polymorphism in Java by creating an abstract Shape class with an abstract area() method and its subclasses Circle, Rectangle, and Triangle, and to calculate the area and total area using a single loop over a mixed Shape[] array.

Code Screenshot:

Shape.java

```java
abstract class Shape {
    abstract double area();
}
```

Circle.java

```java
class Circle extends Shape{
    double radius;
    Circle(double r){
    radius=r;
    }
    double area(){
   return 3.14*radius*radius;
    }
}
```

Rectangle.java

```java
class Rectangle extends Shape{
    double lenght,width;
    Rectangle(double l,double w){
        lenght=l;
        width=w;
    }
    double area(){
        return lenght*width;
    }
}

```

Triangle.java

```java
class Triangle extends Shape{
    double base,hight;
    Triangle(double B, double H) {
            base = B;
            hight = H;
    }
    double area() {
        return base * hight;
    }
}

```

```java
public class Main {
    Run | Debug
    public static void main(String[] args) {
    Shape shapes[]={
            new Circle(r: 5),
            new Rectangle(l: 4,w: 6),
            new Triangle(B: 3,H: 8)
    };
        double total = 0;
        double largest = 0;

        for(Shape s : shapes) {

            double a = s.area();

            System.out.println(a);

            total += a;

            if(a > largest)
                largest = a;
        }

System.out.println("Total: " + total);
System.out.println("Largest: " + largest);
    }
}
```

Output Screenshot:

```
cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 5\1\" ; if ($?) { javac Main.java } ; if ($?) { java Main }
78.5
24.0
24.0
Total: 126.5
Largest: 78.5
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 5\1>
```

Learning Outcome:

- Understand and implement abstract classes and abstract methods.
- Apply inheritance by creating subclasses from an abstract parent class.
- Demonstrate runtime polymorphism using a mixed Shape[] array.
- Override the area() method for different shapes.
- Process different objects using a single polymorphic loop.
- Calculate the running total and identify the shape with the largest area.

Aim: To implement **abstraction, inheritance, constructor chaining, and runtime polymorphism** in Java by creating an abstract Employee class with FullTime, PartTime, and Intern subclasses, each providing its own monthlySalary() implementation.

Code Screenshot:

Employee.java

```java
abstract class Employee {
    String name;
    int id;
    Employee(String name,int id){
    this.name=name;
    this.id=id;
    }
    abstract double monthlySalary();
}
```

FullTime.java

```java
class FullTime extends Employee {

    double salary;

    FullTime(String name, int id, double salary) {
        super(name, id);
        this.salary = salary;
    }

    double monthlySalary() {
        return salary;
    }
}
```

PartTime.java

```java
class PartTime extends Employee {
    int hours;
    double rate;
    PartTime(String name, int id, int hours, double rate) {
        super(name, id);
        this.hours = hours;
        this.rate = rate;
    }

    double monthlySalary() {
        return hours * rate;
    }
}
```

Intern.java

```java
class Intern extends Employee {

    double stipend;

    Intern(String name, int id, double stipend) {
        super(name, id);
        this.stipend = stipend;
    }

    double monthlySalary() {
        return stipend;
    }
}
```

Driver.java

```java
public class Driver {
    Run | Debug
    public static void main(String[] args) {
        Employee Employees[]={
        new FullTime(name: "Harsh", id: 101, salary: 50000),
        new PartTime(name: "Riya", id: 102, hours: 100, rate: 20
        new Intern(name: "Amit", id: 103, stipend: 10000)
};
double total=0;
for(Employee e:Employees){
double salary=e.monthlySalary();
System.out.println(e.name+":"+salary);
total+=salary;
 if (e instanceof Intern) {
        System.out.println(x: "Note: This employee is an Intern
}
System.out.println("Total Salary: "+total);
}
}
```

Output Screenshot:

```
  cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 5
\2\" ; if ($?) { javac Driver.java } ; if ($?) { java Driver }
Harsh:50000.0
Riya:20000.0
Amit:10000.0
Note: This employee is an Intern.
Total Salary: 80000.0
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 5\2>
```

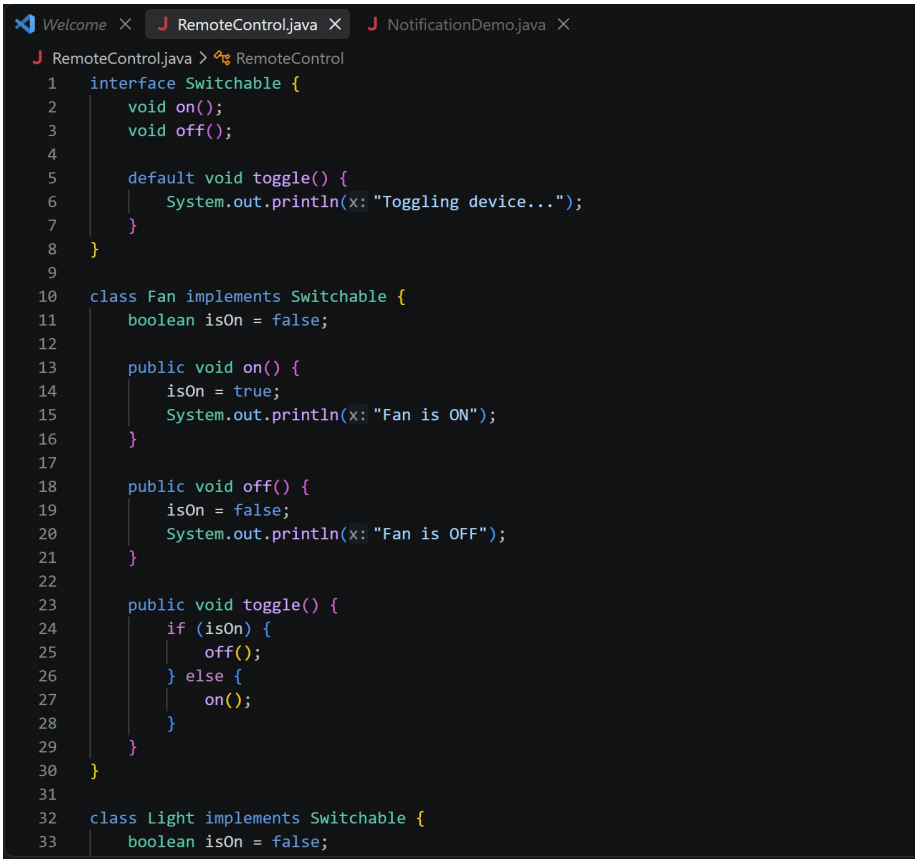
Learning Outcome:

- Understand and implement **abstract classes and abstract methods**.
- Use **inheritance** to create different employee types.
- Use super() to initialize shared employee information such as **name and ID**.
- Demonstrate **runtime polymorphism** using a mixed Employee[] array.
- Calculate different salaries through the same monthlySalary() method.
- Use the **instanceof operator** to identify specific employee types such as interns.
- Calculate and display the total monthly payroll.

PRACTICAL -6

Aim: To implement a remote control system using interfaces, default methods, functional interfaces, anonymous classes, and lambda expressions to control switchable devices such as Fan and Light.

Code Screenshot:



```
    public void on() {
        isOn = true;
        System.out.println(x: "Light is ON");
    }

    public void off() {
        isOn = false;
        System.out.println(x: "Light is OFF");
    }

    public void toggle() {
        if (isOn) {
            off();
        } else {
            on();
        }
    }
}

@FunctionalInterface
interface SwitchRule {
    boolean maySwitchOn(Switchable device, int hour);
}

public class RemoteControl {
    Run | Debug
    public static void main(String[] args) {

        Switchable[] devices = {
            new Fan(),
            new Light()
        };
```

```
        for (Switchable device : devices) {
            device.toggle();
        }

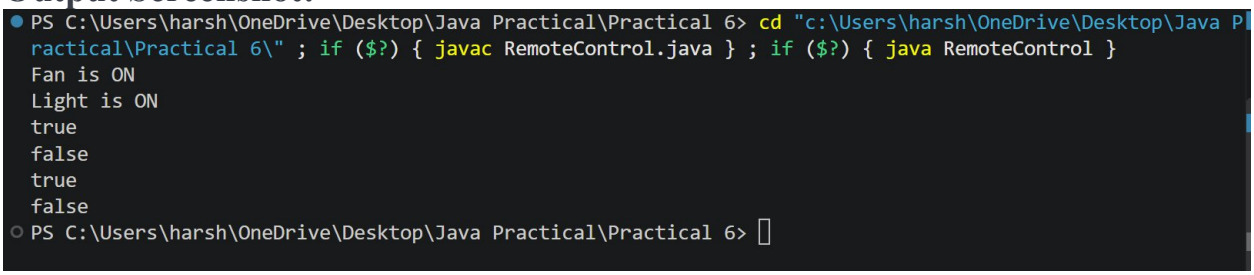
        SwitchRule rule1 = new SwitchRule() {
            public boolean maySwitchOn(Switchable device, int hour) {
                return hour >= 6 && hour <= 22;
            }
        };

        System.out.println(rule1.maySwitchOn(devices[0], hour: 10));
        System.out.println(rule1.maySwitchOn(devices[0], hour: 23));

        SwitchRule rule2 =
            (device, hour) -> hour >= 6 && hour <= 22;

        System.out.println(rule2.maySwitchOn(devices[1], hour: 10));
        System.out.println(rule2.maySwitchOn(devices[1], hour: 23));
    }
}
```

Output Screenshot:

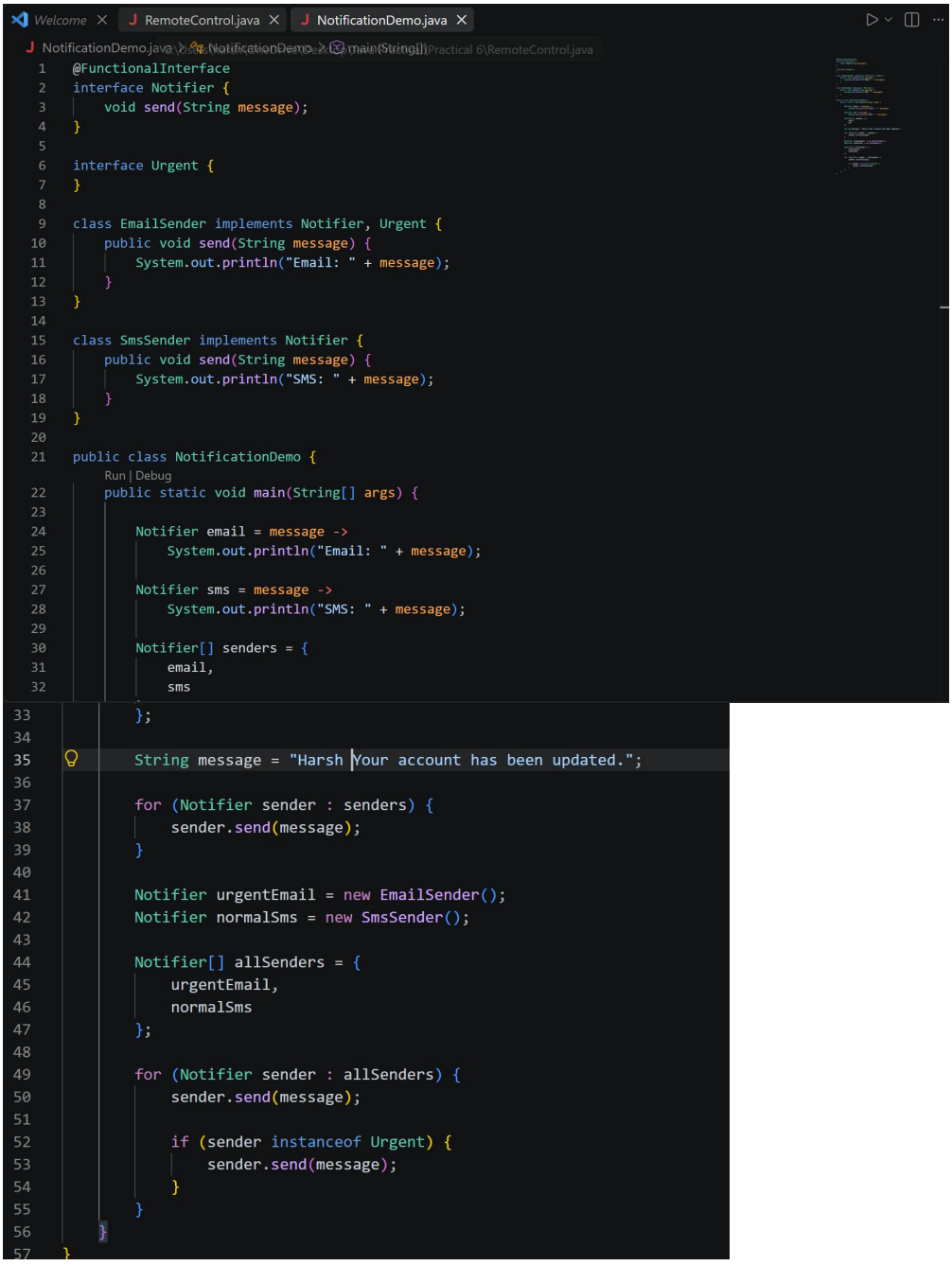


Learning Outcome:

To understand the use of interfaces, default methods, functional interfaces, anonymous classes, lambda expressions, and polymorphism in Java.

Aim: To implement a notification system using functional interfaces, lambda expressions, arrays, and marker interfaces for Email and SMS notification senders.

Code Screenshot:



```java
@FunctionalInterface
interface Notifier {
    void send(String message);
}

interface Urgent {
}

class EmailSender implements Notifier, Urgent {
    public void send(String message) {
        System.out.println("Email: " + message);
    }
}

class SmsSender implements Notifier {
    public void send(String message) {
        System.out.println("SMS: " + message);
    }
}

public class NotificationDemo {
    Run | Debug
    public static void main(String[] args) {

        Notifier email = message ->
            System.out.println("Email: " + message);

        Notifier sms = message ->
            System.out.println("SMS: " + message);

        Notifier[] senders = {
            email,
            sms
        };

        String message = "Harsh Your account has been updated.";

        for (Notifier sender : senders) {
            sender.send(message);
        }

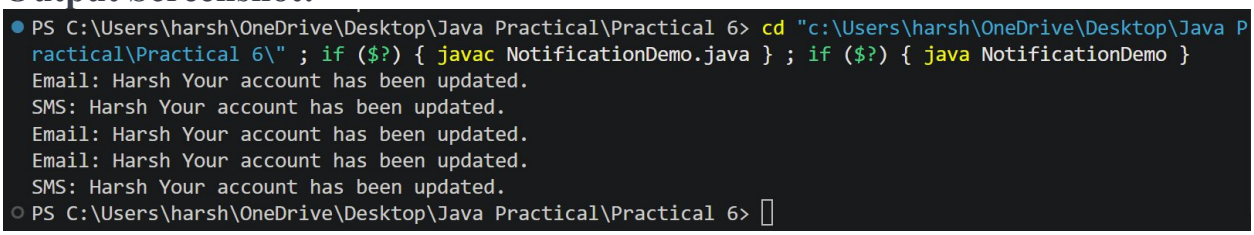
        Notifier urgentEmail = new EmailSender();
        Notifier normalSms = new SmsSender();

        Notifier[] allSenders = {
            urgentEmail,
            normalSms
        };

        for (Notifier sender : allSenders) {
            sender.send(message);

            if (sender instanceof Urgent) {
                sender.send(message);
            }
        }
    }
}
```

Output Screenshot:



Learning Outcome:

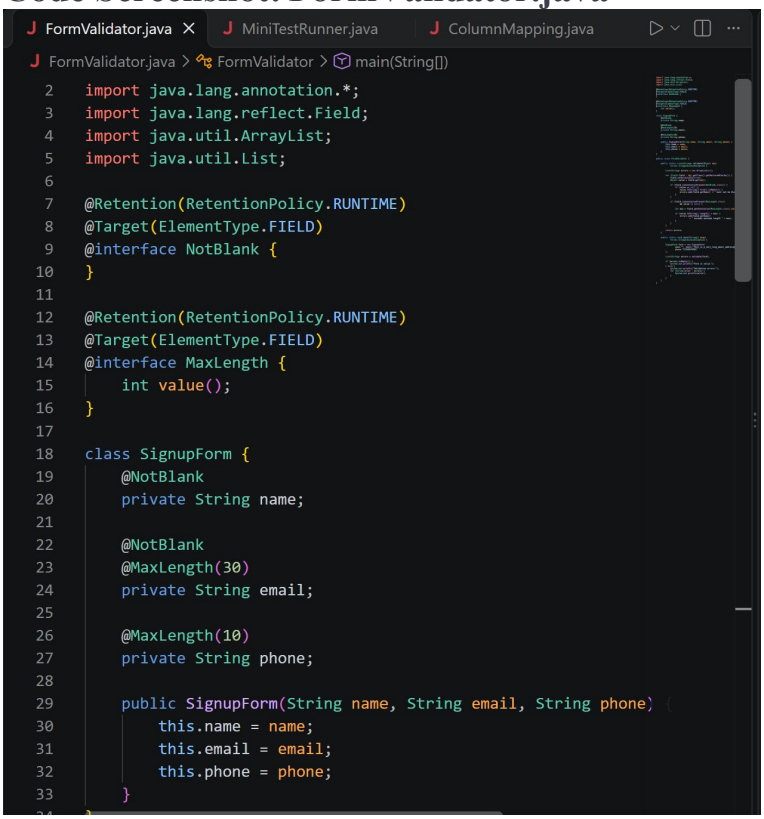
To understand functional interfaces, lambda expressions, marker interfaces, instanceof, and broadcasting messages to multiple objects in Java.

PRACTICAL -7

Aim:

To create custom annotations @NotBlank and @MaxLength and validate form fields using Java Reflection.

Code Screenshot: FormValidator.java



FormValidator.java MiniTestRunner.java ColumnMapping.java

FormValidator.java > FormValidator > main(String[])

```java
public class FormValidator {

    public static List<String> validate(Object obj)
            throws IllegalAccessException {

        List<String> errors = new ArrayList<>();

        for (Field field : obj.getClass().getDeclaredFields()) {
            field.setAccessible(true);
            Object value = field.get(obj);

            if (field.isAnnotationPresent(NotBlank.class)) {
                if (value == null ||
                    value.toString().trim().isEmpty()) {
                    errors.add(field.getName() + " must not be blank");
                }
            }

            if (field.isAnnotationPresent(MaxLength.class)
                    && value != null) {

                int max = field.getAnnotation(MaxLength.class).value();

                if (value.toString().length() > max) {
                    errors.add(field.getName()
                            + " exceeds maximum length " + max);
                }
            }
        }

        return errors;
    }
```

FormValidator.java MiniTestRunner.java ColumnMapping.java

FormValidator.java > FormValidator > main(String[])

```java
public class FormValidator {

    Run | Debug
    public static void main(String[] args)
            throws IllegalAccessException {

        SignupForm form = new SignupForm(
                name: "", email: "this_is_a_very_long_email_address@example.com",
                phone: "12345678901"
        );

        List<String> errors = validate(form);

        if (errors.isEmpty()) {
            System.out.println("Form is valid.");
        } else {
            System.out.println("Validation errors:");
            for (String error : errors) {
                System.out.println(error);
            }
        }
    }
}
```

Output Screenshot

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7\" ; if ($?) { javac FormValidator.java } ; if ($?) { java FormValidator }
Validation errors:
name must not be blank
email exceeds maximum length 30
phone exceeds maximum length 10
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7>
```

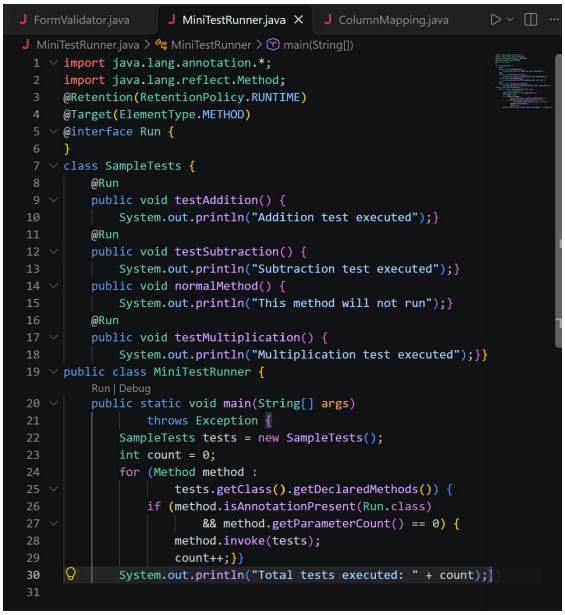
:

Learning Outcome:

Learned to define custom annotations and inspect field values at runtime to identify validation errors.

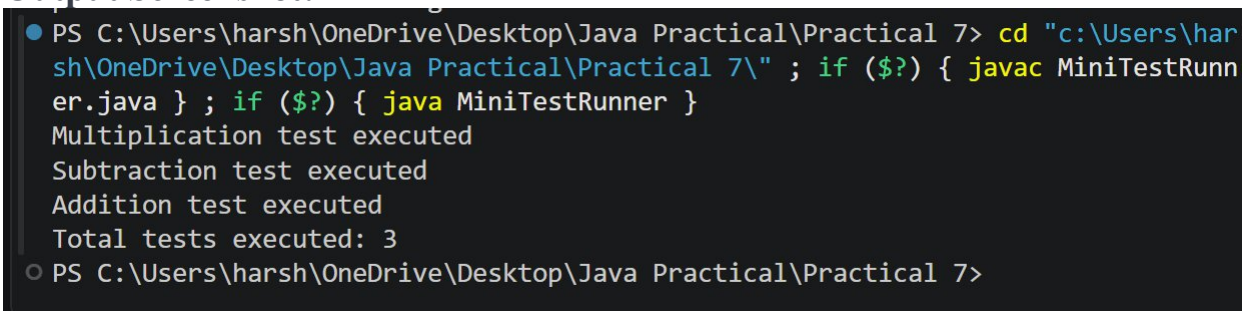
Aim:
To create a custom @Run annotation and execute annotated methods using Reflection.

Code Screenshot: MiniTestRunner.java



```java
import java.lang.annotation.*;
import java.lang.reflect.Method;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.METHOD)
@interface Run {
}
class SampleTests {
    @Run
    public void testAddition() {
        System.out.println("Addition test executed");}
    @Run
    public void testSubtraction() {
        System.out.println("Subtraction test executed");}
    public void normalMethod() {
        System.out.println("This method will not run");}
    @Run
    public void testMultiplication() {
        System.out.println("Multiplication test executed");}}
public class MiniTestRunner {
    public static void main(String[] args)
            throws Exception {
        SampleTests tests = new SampleTests();
        int count = 0;
        for (Method method :
                tests.getClass().getDeclaredMethods()) {
            if (method.isAnnotationPresent(Run.class)
                    && method.getParameterCount() == 0) {
                method.invoke(tests);
                count++;}}
        System.out.println("Total tests executed: " + count);}
```

Output Screenshot:



```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7\" ; if ($?) { javac MiniTestRunner.java } ; if ($?) { java MiniTestRunner }
Multiplication test executed
Subtraction test executed
Addition test executed
Total tests executed: 3
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7>
```

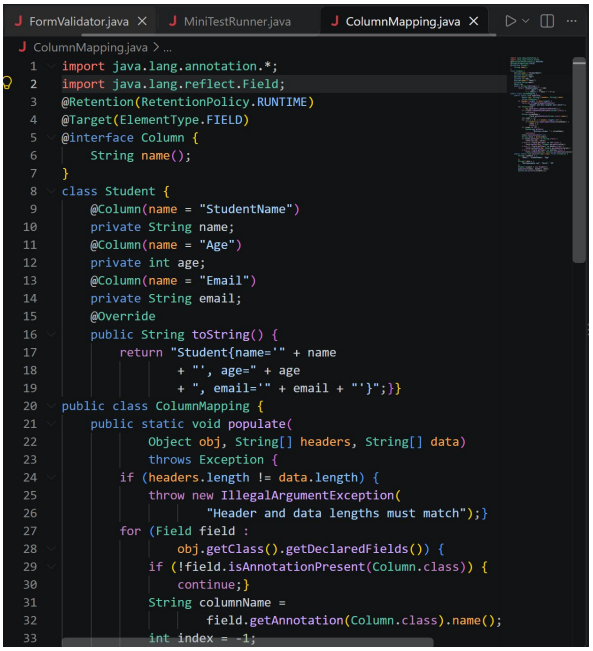
Learning Outcome:
Learned to identify and invoke selected methods dynamically using Reflection and count the executed tests.

Aim

To create a custom @Column annotation and map header and data rows to object fields using Reflection.

:

Code Screenshot:



```java
import java.lang.annotation.*;
import java.lang.reflect.Field;
@Retention(RetentionPolicy.RUNTIME)
@Target(ElementType.FIELD)
@interface Column {
    String name();
}
class Student {
    @Column(name = "StudentName")
    private String name;
    @Column(name = "Age")
    private int age;
    @Column(name = "Email")
    private String email;
    @Override
    public String toString() {
        return "Student{name='" + name
                + "', age=" + age
                + ", email='" + email + "'}";}}
public class ColumnMapping {
    public static void populate(
            Object obj, String[] headers, String[] data)
            throws Exception {
        if (headers.length != data.length) {
            throw new IllegalArgumentException(
                    "Header and data lengths must match");}
        for (Field field :
                obj.getClass().getDeclaredFields()) {
            if (!field.isAnnotationPresent(Column.class)) {
                continue;}
            String columnName =
                    field.getAnnotation(Column.class).name();
            int index = -1;
```

```java
public class ColumnMapping {
    public static void populate(
            for (int i = 0; i < headers.length; i++) {
                if (headers[i].equalsIgnoreCase(columnName)) {
                    index = i;
                    break;}}
            if (index == -1) {
                System.out.println(
                        "Missing column: " + columnName);
                continue;}
            field.setAccessible(true);
            String value = data[index];
            if (field.getType() == String.class) {
                field.set(obj, value);
            } else if (field.getType() == int.class) {
                field.setInt(obj, Integer.parseInt(value));
            } else if (field.getType() == double.class) {
                field.setDouble(obj, Double.parseDouble(value);
            } else if (field.getType() == boolean.class) {
                field.setBoolean(obj, Boolean.parseBoolean(val
    Run | Debug
    public static void main(String[] args) throws Exception {
        String[] headers = {
            "Email", "StudentName", "Age"
        };
        String[] data = {
            "harsh@example.com", "Harsh", "20"
        };
        Student student = new Student();
        populate(student, headers, data);
        System.out.println(student);}}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7\" ; if ($?) { javac ColumnMapping.java } ; if ($?) { java ColumnMapping }
Student{name='Harsh', age=20, email='harsh@example.com'}
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 7>
```

Learning Outcome:

Learned to match column names with object fields and populate objects dynamically, including handling missing columns.

PRACTICAL -8

Aim:

Guarded Calculator: To implement a calculator using custom DivideByZeroException, separate handling for invalid input, looping until successful execution, and finally for logging attempts.

Code Screenshot:

TryWithResourcesDemo.java > ExamSession

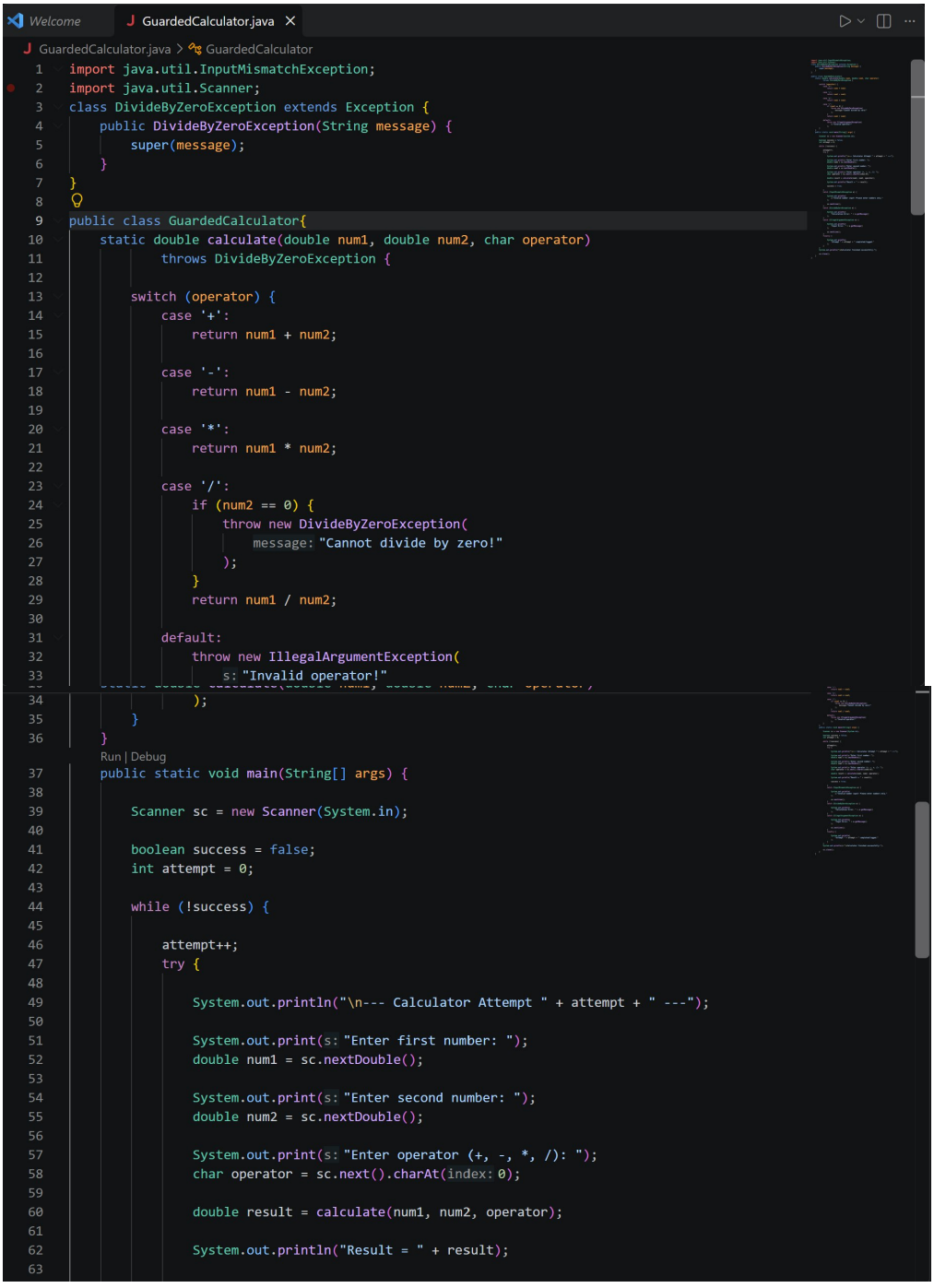
```java
class ExamSession implements AutoCloseable {
    private String studentName;
    public ExamSession(String studentName) {
        this.studentName = studentName;
        System.out.println(
            "Exam session opened for " + studentName
        );
    }
    public void startExam() {
        System.out.println(
            "Exam started for " + studentName
        );
    }
    @Override
    public void close() {

        System.out.println(
            "Exam session closed for " + studentName
        );}}
public class TryWithResourcesDemo{
    Run | Debug
    public static void main(String[] args) {
        try (ExamSession session =
                new ExamSession(studentName: "Harsh")) {
            session.startExam();
            System.out.println(
                x: "Doing exam work...");
            throw new RuntimeException(
                message: "Something went wrong during the exam!");}
        catch (RuntimeException e) {
            System.out.println(
                "Caught error: " + e.getMessage());}}}
```

TryWithResourcesDemo.java > ExamSession

```java
class ExamSession implements AutoCloseable {
    private String studentName;
    public ExamSession(String studentName) {
        this.studentName = studentName;
        System.out.println(
            "Exam session opened for " + studentName
        );
    }
    public void startExam() {
        System.out.println(
            "Exam started for " + studentName
        );
    }
    @Override
    public void close() {

        System.out.println(
            "Exam session closed for " + studentName
        );}}
public class TryWithResourcesDemo{
    Run | Debug
    public static void main(String[] args) {
        try (ExamSession session =
                new ExamSession(studentName: "Harsh")) {
            session.startExam();
            System.out.println(
                x: "Doing exam work...");
            throw new RuntimeException(
                message: "Something went wrong during the exam!");}
        catch (RuntimeException e) {
            System.out.println(
                "Caught error: " + e.getMessage());}}}
```



```java
import java.util.InputMismatchException;
import java.util.Scanner;
class DivideByZeroException extends Exception {
    public DivideByZeroException(String message) {
        super(message);
    }
}

public class GuardedCalculator{
    static double calculate(double num1, double num2, char operator)
            throws DivideByZeroException {

        switch (operator) {
            case '+':
                return num1 + num2;

            case '-':
                return num1 - num2;

            case '*':
                return num1 * num2;

            case '/':
                if (num2 == 0) {
                    throw new DivideByZeroException(
                        message: "Cannot divide by zero!"
                    );
                }
                return num1 / num2;

            default:
                throw new IllegalArgumentException(
                    s: "Invalid operator!"
                );
        }
    }
    Run | Debug
    public static void main(String[] args) {

        Scanner sc = new Scanner(System.in);

        boolean success = false;
        int attempt = 0;

        while (!success) {

            attempt++;
            try {

                System.out.println("\n--- Calculator Attempt " + attempt + " ---");

                System.out.print(s: "Enter first number: ");
                double num1 = sc.nextDouble();

                System.out.print(s: "Enter second number: ");
                double num2 = sc.nextDouble();

                System.out.print(s: "Enter operator (+, -, *, /): ");
                char operator = sc.next().charAt(index: 0);

                double result = calculate(num1, num2, operator);

                System.out.println("Result = " + result);

```



```
                    x: "Invalid number input! Please enter numbers only."
                );

                sc.nextLine();
            }
            catch (DivideByZeroException e) {

                System.out.println(
                    "Calculation Error: " + e.getMessage()
                );
            }
            catch (IllegalArgumentException e) {

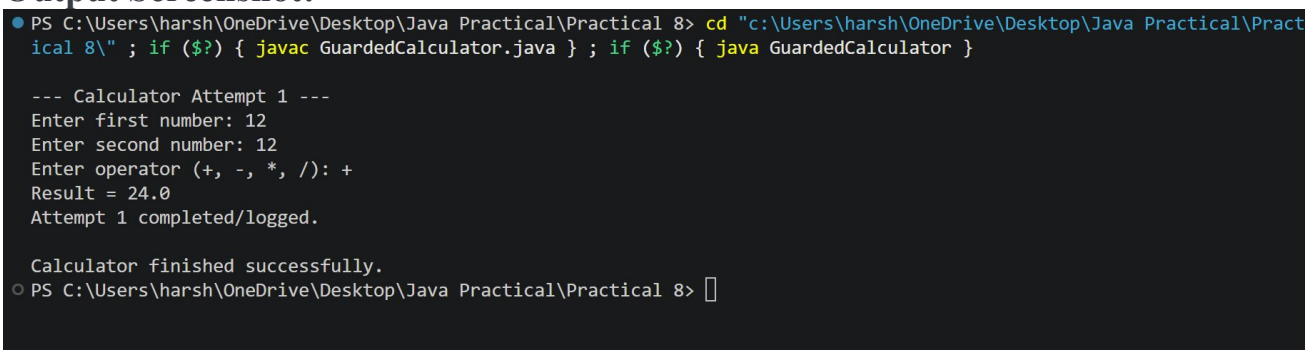
                System.out.println(
                    "Input Error: " + e.getMessage()
                );

                sc.nextLine();
            }
            finally {

                System.out.println(
                    "Attempt " + attempt + " completed/logged."
                );
            }
        }
        System.out.println(x: "\nCalculator finished successfully.");

        sc.close();
    }
}
```

Output Screenshot:



Learning Outcome:

To implement custom exception handling for divide-by-zero and invalid input using try-catch-finally.

Aim: Stock Issue: To implement a warehouse stock-issue system using custom checked exceptions for invalid quantities and insufficient stock, while processing multiple requests without terminating the program.

Code Screenshot:



```java
import java.util.*;

class OutOfStockException extends Exception {

    private int shortfall;

    public OutOfStockException(String message, int shortfall) {
        super(message);
        this.shortfall = shortfall;
    }

    public int getShortfall() {
        return shortfall;
    }
}

class InvalidQuantityException extends Exception {

    public InvalidQuantityException(String message) {
        super(message);
    }
}
class Warehouse {
    private Map<String, Integer> stock;
    public Warehouse() {

        stock = new HashMap<>();
        stock.put(key: "Laptop", value: 10);
        stock.put(key: "Mouse", value: 5);
        stock.put(key: "Keyboard", value: 8);
        stock.put(key: "Monitor", value: 3);
    }
    public void issue(String item, int qty)
            throws OutOfStockException, InvalidQuantityException {

        if (qty <= 0) {
            throw new InvalidQuantityException(
                message: "Quantity must be greater than 0."
            ); }

        if (!stock.containsKey(item)) {
            System.out.println(
                "Item '" + item + "' does not exist."
            );
            return;}

        int available = stock.get(item);
        if (qty > available) {
            int shortfall = qty - available;
            throw new OutOfStockException(
                "Not enough stock for " + item,
                shortfall
            );
        }

        stock.put(item, available - qty);

        System.out.println(
            "Issued " + qty + " " + item +
            "(s) successfully."
        );
    }
    public void displayStock() {
```

```java
        System.out.println(x: "\n--- Current Stock ---");

        for (Map.Entry<String, Integer> entry : stock.entrySet()) {
            System.out.println(
                entry.getKey() + " : " +
                entry.getValue()
            );}}}

public class  StockIssue{

    Run | Debug
    public static void main(String[] args) {
        Warehouse warehouse = new Warehouse();

        String[][] requests = {
            {"Laptop", "3"},
            {"Mouse", "10"},
            {"Keyboard", "-2"},
            {"Monitor", "2"},
            {"Laptop", "20"},
            {"Mouse", "2"}
        };


        for (String[] request : requests) {

            String item = request[0];
            int qty = Integer.parseInt(request[1]);

            System.out.println(
                "\nRequest: " + item + " x " + qty
                try {
                    warehouse.issue(item, qty);
                }
                catch (OutOfStockException e) {
                    System.out.println(
                        "Out of Stock: " +
                        e.getMessage()
                    );

                    System.out.println(
                        "Shortfall: " +
                        e.getShortfall()
                    );
                }
                catch (InvalidQuantityException e) {
                    System.out.println(
                        "Invalid Quantity: " +
                        e.getMessage()
                    );
                }
            }
            warehouse.displayStock();
        }
    }
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 8> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 8\" ; if ($?) { javac StockIssue.java } ; if ($?) { java StockIssue }

Request: Laptop x 3
Issued 3 Laptop(s) successfully.

Request: Mouse x 10
Out of Stock: Not enough stock for Mouse
Shortfall: 5
```

Learning Outcome:

To implement custom checked exceptions for invalid quantities and insufficient stock while processing multiple requests safely.

Aim: AutoCloseable Resource: To understand try-with-resources by creating an AutoCloseable resource and verifying that it is automatically closed even when an exception occurs.

Code Screenshot:

TryWithResourcesDemo.java > ExamSession

```java
class ExamSession implements AutoCloseable {
    private String studentName;
    public ExamSession(String studentName) {
        this.studentName = studentName;
        System.out.println(
            "Exam session opened for " + studentName
        );
    }
    public void startExam() {
        System.out.println(
            "Exam started for " + studentName
        );
    }
    @Override
    public void close() {

        System.out.println(
            "Exam session closed for " + studentName
        );}}
public class TryWithResourcesDemo{
    Run | Debug
    public static void main(String[] args) {
        try (ExamSession session =
                new ExamSession(studentName: "Harsh")) {
            session.startExam();
            System.out.println(
                x: "Doing exam work...");
            throw new RuntimeException(
                message: "Something went wrong during the exam!");}
        catch (RuntimeException e) {
            System.out.println(
                "Caught error: " + e.getMessage());}}}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 8> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 8\" ; if ($?) { javac TryWithResourcesDemo.java } ; if ($?) { java TryWithResourcesDemo }
Exam session opened for Harsh
Exam started for Harsh
Doing exam work...
Exam session closed for Harsh
Caught error: Something went wrong during the exam!
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 8>
```

Learning Outcome:

To implement AutoCloseable and use try-with-resources for automatic resource closure during exception handling.

PRACTICAL -9

Aim:

Counter Race:To demonstrate a race condition using multiple threads and fix it using synchronization.

Code :

```
class Counter {
   int count = 0;

   void increment() {
      count++;
   }

   synchronized void synchronizedIncrement() {
      count++;
   }
}

public class CounterRace {
   public static void main(String[] args) throws InterruptedException {
      Counter c1 = new Counter();
      Thread[] threads = new Thread[10];

      for (int i = 0; i < 10; i++) {
         threads[i] = new Thread(() -> {
            for (int j = 0; j < 100000; j++) {
               c1.increment();
            }
         });
         threads[i].start();
      }

      for (Thread t : threads) {
         t.join();
      }

      System.out.println("Without synchronization: " + c1.count);

      Counter c2 = new Counter();

      for (int i = 0; i < 10; i++) {
         threads[i] = new Thread(() -> {
            for (int j = 0; j < 100000; j++) {
               c2.synchronizedIncrement();
            }
```

```
            });
            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }

        System.out.println("With synchronization: " + c2.count);
    }
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 9> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 9\" ; if ($?) { javac CounterRace.java } ; if ($?) { java CounterRace }
Without synchronization: 902014
With synchronization: 1000000
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 9>
```

Learning Outcome:

To understand race conditions and use the synchronized keyword to safely update shared data.

Aim:

To demonstrate a race condition in a shared seat-booking system and prevent overselling using synchronization.

Code :

```
class TicketCounter {
   int seatsLeft = 5;

   void book(String name) {
      if (seatsLeft > 0) {
         try {
            Thread.sleep(10);
         } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
         }
         seatsLeft--;
         System.out.println(name + " booked a seat");
      }
   }

   synchronized void synchronizedBook(String name) {
      if (seatsLeft > 0) {
         try {
            Thread.sleep(10);
         } catch (InterruptedException e) {
            Thread.currentThread().interrupt();
         }
         seatsLeft--;
         System.out.println(name + " booked a seat");
      }
   }
}

public class SeatBookingRace {
   public static void main(String[] args) throws InterruptedException {
      TicketCounter c1 = new TicketCounter();
      Thread[] threads = new Thread[10];

      for (int i = 0; i < 10; i++) {
         String name = "User-" + (i + 1);
         threads[i] = new Thread(() -> c1.book(name));
         threads[i].start();
      }
```

```
        for (Thread t : threads) {
            t.join();
        }

        System.out.println("Seats left without synchronization: " + c1.seatsLeft);

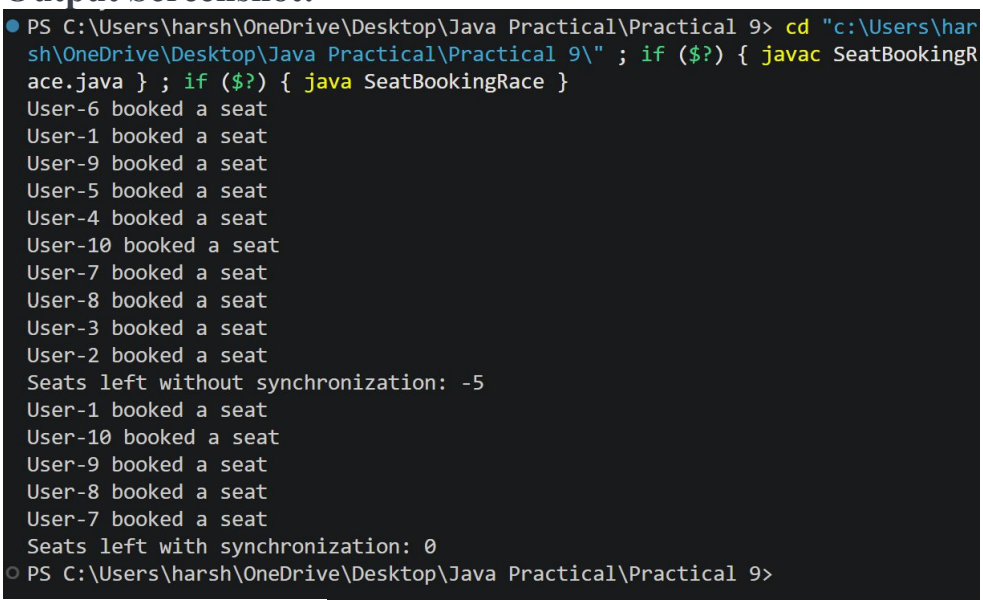
        TicketCounter c2 = new TicketCounter();

        for (int i = 0; i < 10; i++) {
            String name = "User-" + (i + 1);
            threads[i] = new Thread(() -> c2.synchronizedBook(name));
            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }

        System.out.println("Seats left with synchronization: " + c2.seatsLeft);
    }
}
```

Output Screenshot:



Learning Outcome:

To understand thread safety in shared resources and use synchronized methods to ensure that only available seats are booked.

Aim:

To demonstrate incorrect results caused by multiple threads updating a shared total and compare two thread-safe solutions.

Code :

```
class SharedTotal {
   long total = 0;

   void add(long value) {
      total += value;
   }

   synchronized void synchronizedAdd(long value) {
      total += value;
   }
}

public class ArrayThreadSum {
   static final int SIZE = 1000000;
   static final int THREADS = 4;

   public static void main(String[] args) throws InterruptedException {
      int[] arr = new int[SIZE];

      for (int i = 0; i < SIZE; i++) {
         arr[i] = 1;
      }

      SharedTotal unsafe = new SharedTotal();
      Thread[] threads = new Thread[THREADS];

      int part = SIZE / THREADS;

      long start = System.nanoTime();

      for (int i = 0; i < THREADS; i++) {
         int startIndex = i * part;
         int endIndex = (i + 1) * part;

         threads[i] = new Thread(() -> {
            for (int j = startIndex; j < endIndex; j++) {
               unsafe.add(arr[j]);
            }
         });
```

```
            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }

        long end = System.nanoTime();

        System.out.println("Wrong shared total: " + unsafe.total);
        System.out.println("Time: " + (end - start) / 1000000.0 + " ms");

        SharedTotal safe = new SharedTotal();

        start = System.nanoTime();

        for (int i = 0; i < THREADS; i++) {
            int startIndex = i * part;
            int endIndex = (i + 1) * part;

            threads[i] = new Thread(() -> {
                for (int j = startIndex; j < endIndex; j++) {
                    safe.synchronizedAdd(arr[j]);
                }
            });

            threads[i].start();
        }

        for (Thread t : threads) {
            t.join();
        }

        end = System.nanoTime();

        System.out.println("Synchronized total: " + safe.total);
        System.out.println("Time: " + (end - start) / 1000000.0 + " ms");
    }
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 9> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 9\" ; if ($?) { javac ArrayThreadSum.java } ; if ($?) { java ArrayThreadSum }
Wrong shared total: 498972
Time: 21.2696 ms
Synchronized total: 1000000
Time: 62.7545 ms
```

Learning Outcome:

To understand race conditions in parallel computation and compare synchronization with independent partial totals.

PRACTICAL -10

Aim: To implement a fixed thread pool using ExecutorService in Java to execute multiple tasks concurrently and observe thread reuse.

Code Screenshot:



```java
import java.util.concurrent.ExecutorService;
import java.util.concurrent.Executors;
import java.util.concurrent.TimeUnit;

public class ThreadPoolRunner {
    Run | Debug
    public static void main(String[] args) throws InterruptedException {

        ExecutorService pool = Executors.newFixedThreadPool(nThreads: 3);

        for (int i = 1; i <= 10; i++) {
            final int id = i;

            pool.submit(() -> {
                System.out.println("Task " + id + " is running by "
                        + Thread.currentThread().getName());

                try {
                    Thread.sleep(millis: 500);
                } catch (InterruptedException e) {
                    Thread.currentThread().interrupt();
                }
            });
        }

        pool.shutdown();
        pool.awaitTermination(timeout: 1, TimeUnit.MINUTES);

        System.out.println(x: "All tasks completed.");
    }
}
```

Output Screenshot:



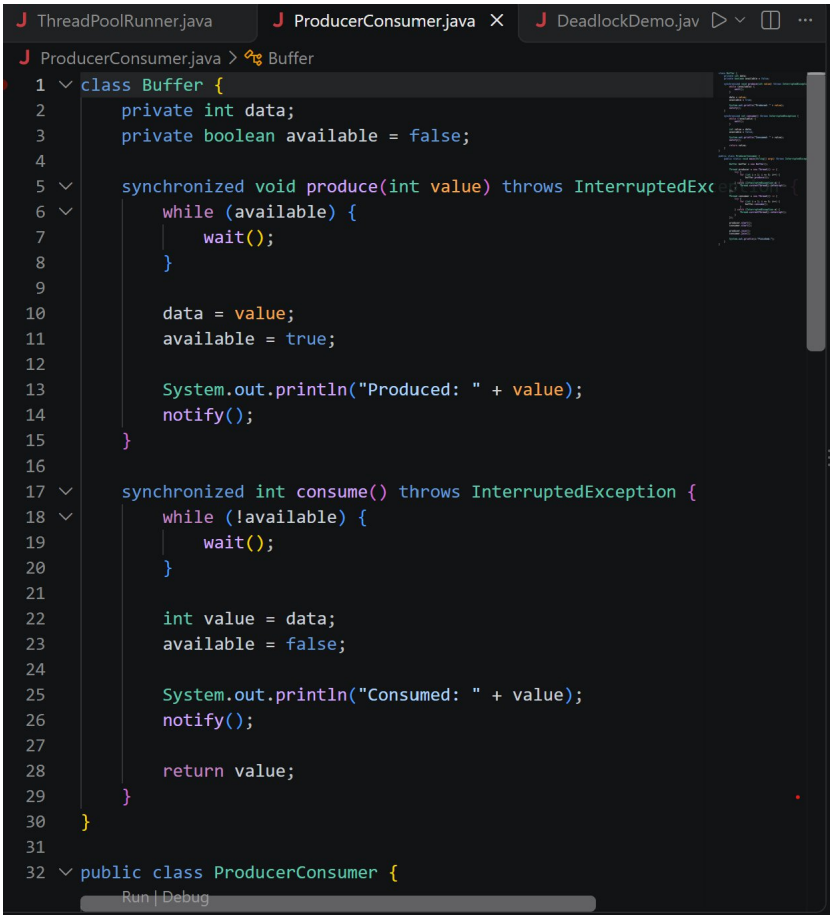
```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 10> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 10\" ; if ($?) { javac ThreadPoolRunner.java } ; if ($?) { java ThreadPoolRunner }
Task 3 is running by pool-1-thread-3
Task 2 is running by pool-1-thread-2
Task 1 is running by pool-1-thread-1
Task 5 is running by pool-1-thread-3
Task 4 is running by pool-1-thread-2
Task 6 is running by pool-1-thread-1
Task 7 is running by pool-1-thread-3
Task 8 is running by pool-1-thread-1
Task 9 is running by pool-1-thread-2
Task 10 is running by pool-1-thread-3
All tasks completed.
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 10>
```

Learning Outcome:

Understand the concept of multithreading and concurrent task execution.
Learn to create a fixed thread pool using Executors.newFixedThreadPool().
Understand task submission, thread reuse, and pool shutdown using shutdown() and awaitTermination().

Aim: To implement the Producer–Consumer problem in Java using threads and synchronization methods such as wait() and notify() to coordinate access to a shared buffer.

Code Screenshot:



```java
public class ProducerConsumer {
    public static void main(String[] args) throws InterruptedE
        Buffer buffer = new Buffer();

        Thread producer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.produce(i);
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        Thread consumer = new Thread(() -> {
            try {
                for (int i = 1; i <= 5; i++) {
                    buffer.consume();
                }
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        });

        producer.start();
        consumer.start();

        producer.join();
        consumer.join();

        System.out.println(x: "Finished.");
    }
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 10> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 10\" ; if ($?) { javac ProducerConsumer.java } ; if ($?) { java ProducerConsumer }
Produced: 1
Consumed: 1
Produced: 2
Consumed: 2
Produced: 3
Consumed: 3
Produced: 4
Consumed: 4
Produced: 5
Consumed: 5
Finished.
```

Learning Outcome:

Understand inter-thread communication in Java.Learn to use synchronized, wait(), and notify() for thread coordination.Ensure proper synchronization while producing and consuming data without losing item

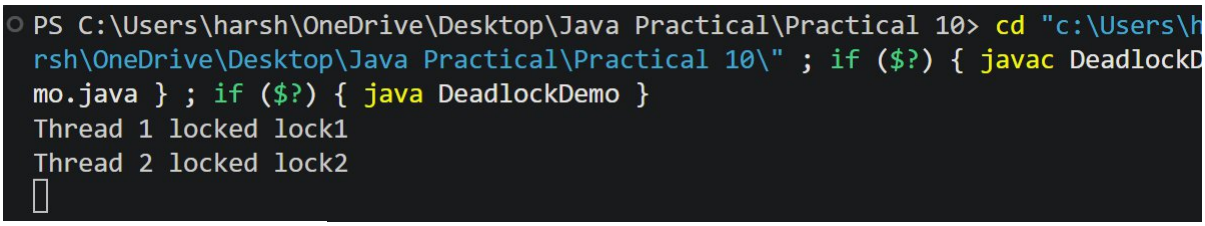
Aim: To demonstrate the deadlock problem in Java by creating two threads that acquire locks in opposite orders and prevent deadlock by following a consistent lock acquisition order.

Code Screenshot:



```
readPoolRunner.java | ProducerConsumer.java | DeadlockDemo.java
DeadlockDemo.java > DeadlockDemo
1  public class DeadlockDemo {
2
3      static final Object lock1 = new Object();
4      static final Object lock2 = new Object();
5
       Run | Debug
6      public static void main(String[] args) {
7
8          Thread t1 = new Thread(() -> {
9              synchronized (lock1) {
10                 System.out.println(x: "Thread 1 locked lock1");
11
12                 try {
13                     Thread.sleep(millis: 100);
14                 } catch (InterruptedException e) {
15                     Thread.currentThread().interrupt();
16                     return;
17                 }
18
19                 synchronized (lock2) {
20                     System.out.println(x: "Thread 1 locked loc
21                 }
22             }
23         });
24
25         Thread t2 = new Thread(() -> {
26             synchronized (lock2) {
27                 System.out.println(x: "Thread 2 locked lock2");
28
29                 try {
30                     Thread.sleep(millis: 100);
31                 } catch (InterruptedException e) {
32                     Thread.currentThread().interrupt();
26             synchronized (lock2) {
27                 System.out.println(x: "Thread 2 locked lock2");
28
29                 try {
30                     Thread.sleep(millis: 100);
31                 } catch (InterruptedException e) {
32                     Thread.currentThread().interrupt();
33                     return;
34                 }
35
36                 synchronized (lock1) {
37                     System.out.println(x: "Thread 2 locked loc
38                 }
39             }
40         });
41
42         t1.start();
43         t2.start();
44     }
45 }
```

Output Screenshot:



Learning Outcome:

Understand the concept and causes of deadlock in multithreaded programs.

Learn how circular waiting can occur when multiple threads compete for resources.

Apply a consistent lock ordering strategy to prevent deadlocks.

PRACTICAL – 11

Problem 1: Save/Load Objects

Aim: To serialize and deserialize an array of Java objects using ObjectOutputStream and ObjectInputStream, and demonstrate the use of the transient keyword.

Code Screenshot:

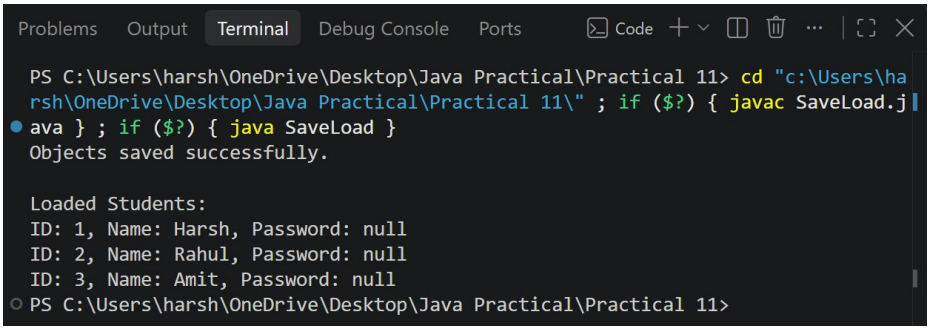


```java
import java.io.*;
class Student implements Serializable {
    private static final long serialVersionUID = 1L;
    int id;
    String name;
    transient String password;
    Student(int id, String name, String password) {
        this.id = id;
        this.name = name;
        this.password = password;
    }
    @Override
    public String toString() {
        return "ID: " + id
                + ", Name: " + name
                + ", Password: " + password;
    }
}
public class SaveLoad {
    Run | Debug
    public static void main(String[] args) {
        Student[] students = {
            new Student(id: 1, name: "Harsh", password: "abc123"),
            new Student(id: 2, name: "Rahul", password: "xyz456"),
            new Student(id: 3, name: "Amit", password: "pass789")
        };
        try (ObjectOutputStream out =
                    new ObjectOutputStream(
                            new FileOutputStream("students.da
            out.writeObject(students);
            System.out.println("Objects saved successfully.");
        } catch (IOException e) {
            System.out.println(e.getMessage());
```



```java
public class SaveLoad {
    public static void main(String[] args) {
            return;
        }
        try (ObjectInputStream in =
                    new ObjectInputStream(
                            new FileInputStream("students.dat
            Student[] loaded =
                    (Student[]) in.readObject();
            System.out.println("\nLoaded Students:");
            for (Student student : loaded) {
                System.out.println(student);
            }
        } catch (IOException | ClassNotFoundException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output Screenshot:

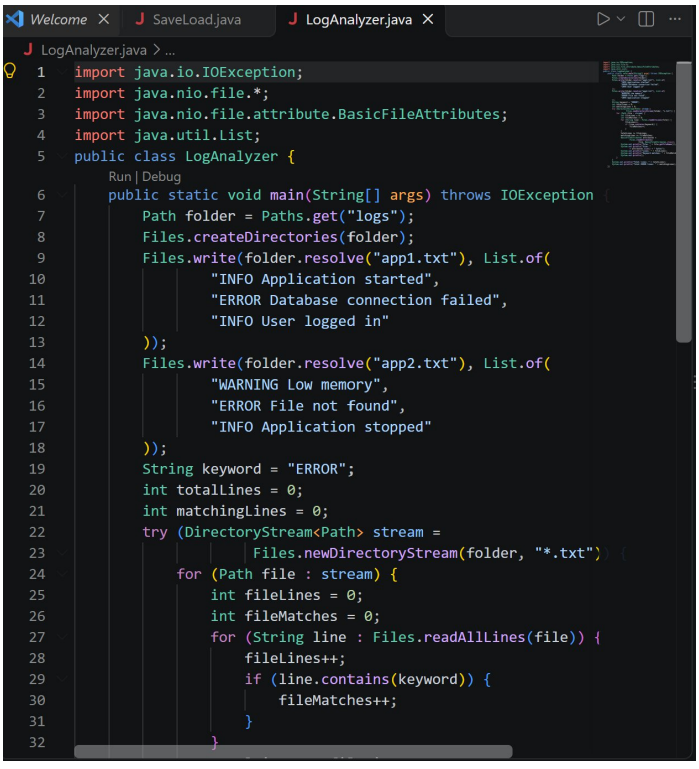


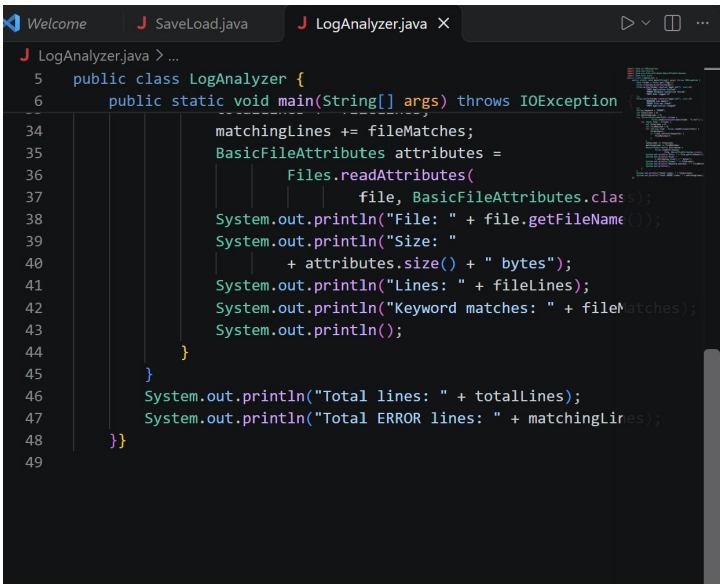
Learning Outcome: Learned to save and restore Java objects using serialization and understood how transient fields are excluded from serialization.

Problem 2: Log Analyzer

Aim: To read multiple text files using Java NIO Path and Files, count total lines and keyword matches, and display file sizes.

Code Screenshot:





```java
public class LogAnalyzer {
    public static void main(String[] args) throws IOException
                matchingLines += fileMatches;
                BasicFileAttributes attributes =
                        Files.readAttributes(
                                file, BasicFileAttributes.class);
                System.out.println("File: " + file.getFileName());
                System.out.println("Size: "
                        + attributes.size() + " bytes");
                System.out.println("Lines: " + fileLines);
                System.out.println("Keyword matches: " + fileMatches);
                System.out.println();
            }
        }
        System.out.println("Total lines: " + totalLines);
        System.out.println("Total ERROR lines: " + matchingLines);
    }}
```

Output Screenshot:



```
cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 11\" ; if ($?) { javac LogAnalyzer.java } ; if ($?) { java LogAnalyzer }
File: app1.txt
Size: 81 bytes
Lines: 3
Keyword matches: 1

File: app2.txt
Size: 68 bytes
Lines: 3
Keyword matches: 1

Total lines: 6
Total ERROR lines: 2
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 11>
```

Learning Outcome: Learned to process text files line by line and retrieve file attributes using Java NIO.

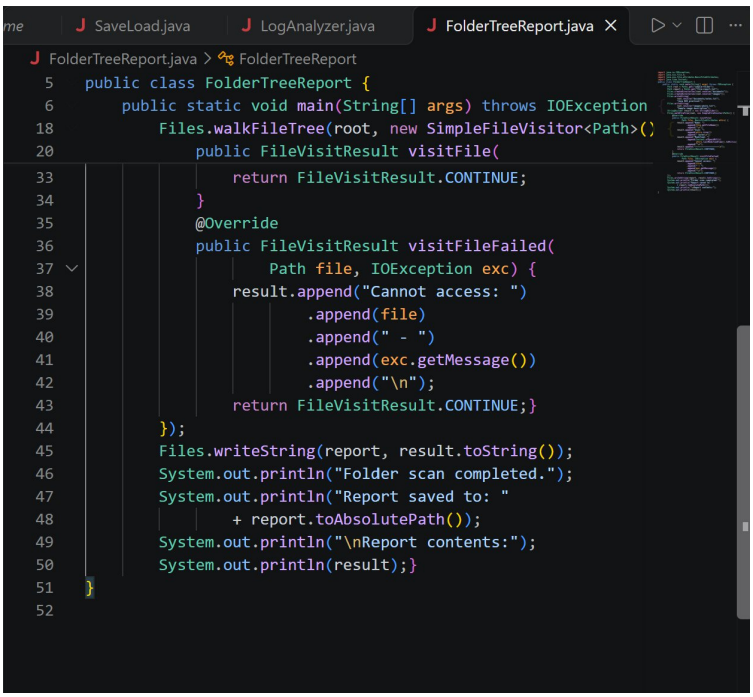
Problem 3: Folder Tree Report (Advanced)

Aim: To traverse a folder tree and generate a report containing file names, sizes, and modification times.

Code Screenshot:




```java
import java.io.IOException;
import java.nio.file.*;
import java.nio.file.attribute.BasicFileAttributes;
import java.time.Instant;
public class FolderTreeReport {
    public static void main(String[] args) throws IOException
        Path root = Paths.get("sample-folder");
        Path report = Paths.get("file-report.txt");
        Files.createDirectories(root.resolve("documents"));
        Files.createDirectories(root.resolve("images"));
        Files.writeString(
                root.resolve("documents/notes.txt"),
                "Java NIO practical");
        Files.writeString(
                root.resolve("images/photo.txt"),
                "Sample image description");
        StringBuilder result = new StringBuilder();
        Files.walkFileTree(root, new SimpleFileVisitor<Path>() {
            @Override
            public FileVisitResult visitFile(
                    Path file, BasicFileAttributes attrs) {
                result.append("Name: ")
                        .append(file.getFileName())
                        .append("\n");
                result.append("Size: ")
                        .append(attrs.size())
                        .append(" bytes\n");
                result.append("Modified: ")
                        .append(Instant.ofEpochMilli(
                                attrs.lastModifiedTime().toMil
                        .append("\n");
                result.append("--------------------\n");
```




```java
public class FolderTreeReport {
    public static void main(String[] args) throws IOException
        Files.walkFileTree(root, new SimpleFileVisitor<Path>()
            public FileVisitResult visitFile(
                return FileVisitResult.CONTINUE;
            }
            @Override
            public FileVisitResult visitFileFailed(
                    Path file, IOException exc) {
                result.append("Cannot access: ")
                        .append(file)
                        .append(" - ")
                        .append(exc.getMessage())
                        .append("\n");
                return FileVisitResult.CONTINUE;}
        });
        Files.writeString(report, result.toString());
        System.out.println("Folder scan completed.");
        System.out.println("Report saved to: "
                + report.toAbsolutePath());
        System.out.println("\nReport contents:");
        System.out.println(result);}
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 11> cd "c:\Users\ha
rsh\OneDrive\Desktop\Java Practical\Practical 11\" ; if ($?) { javac FolderTree
Report.java } ; if ($?) { java FolderTreeReport }
Folder scan completed.
Report saved to: C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 11\fi
le-report.txt

Report contents:
Name: notes.txt
Size: 18 bytes
Modified: 2026-10-27T15:06:38.584Z
--------------------
Name: photo.txt
Size: 24 bytes
Modified: 2026-10-27T15:06:38.586Z
--------------------
```

Learning Outcome: Learned to traverse directories recursively and generate file reports using Java NIO.

PRACTICAL – 12

Problem 1: Word Frequency

Aim: To count the frequency of words in a paragraph using HashMap and sort the words by frequency using Comparator.

Code Screenshot:

```java
import java.util.*;
public class WordFrequency {
    public static void main(String[] args) {
        String text = "java is easy java is powerful java programming";

        String[] words = text.toLowerCase().split("\\W+");
        HashMap<String, Integer> frequency = new HashMap<>();

        for (String word : words) {
            frequency.put(word,
                    frequency.getOrDefault(word, 0) + 1);
        }

        List<Map.Entry<String, Integer>> list =
                new ArrayList<>(frequency.entrySet());

        list.sort(
                Comparator.comparingInt(
                        (Map.Entry<String, Integer> e) -> e.getValue()
                ).reversed()
                .thenComparing(Map.Entry::getKey)
        );

        System.out.println("Word Frequencies:");

        for (Map.Entry<String, Integer> entry : list) {
            System.out.println(
                    entry.getKey() + " : " + entry.getValue());
        }
    }
}
```

Output Screenshot:

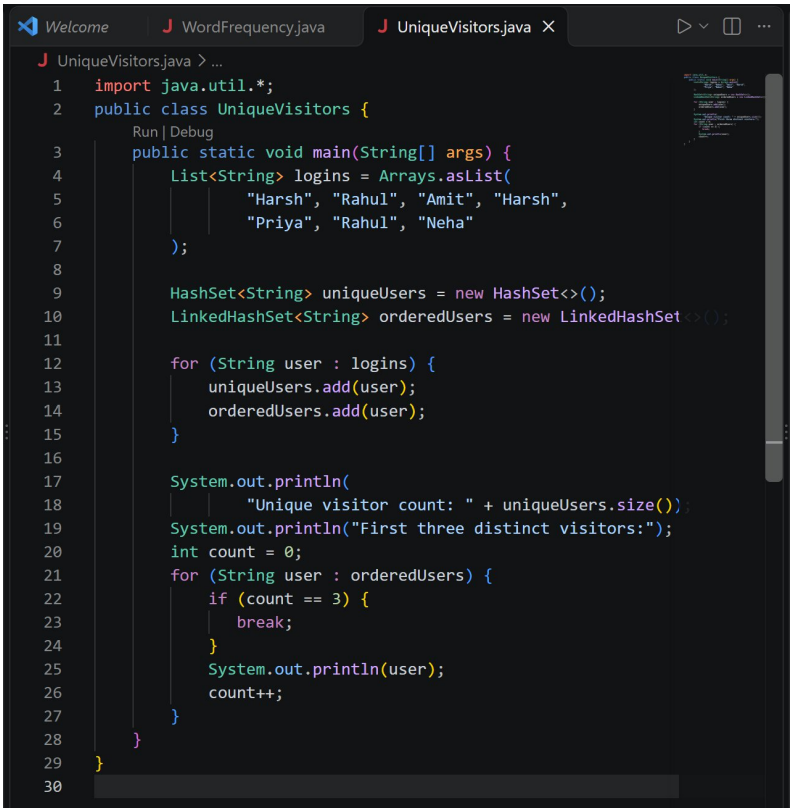
```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12\" ; if ($?) { javac WordFrequency.java } ; if ($?) { java WordFrequency }
Word Frequencies:
java : 3
is : 2
easy : 1
powerful : 1
programming : 1
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12>
```

Learning Outcome: Learned to store word frequencies using HashMap and sort entries in descending order of frequency.

Problem 2: Unique Visitors

Aim: To identify unique login names using HashSet and maintain insertion order using LinkedHashSet.

Code Screenshot:



```java
import java.util.*;
public class UniqueVisitors {
    Run | Debug
    public static void main(String[] args) {
        List<String> logins = Arrays.asList(
                "Harsh", "Rahul", "Amit", "Harsh",
                "Priya", "Rahul", "Neha"
        );

        HashSet<String> uniqueUsers = new HashSet<>();
        LinkedHashSet<String> orderedUsers = new LinkedHashSet<>();

        for (String user : logins) {
            uniqueUsers.add(user);
            orderedUsers.add(user);
        }

        System.out.println(
                "Unique visitor count: " + uniqueUsers.size());
        System.out.println("First three distinct visitors:");
        int count = 0;
        for (String user : orderedUsers) {
            if (count == 3) {
               break;
            }
            System.out.println(user);
            count++;
        }
    }
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12\" ; if ($?) { javac UniqueVisitors.java } ; if ($?) { java UniqueVisitors }
Unique visitor count: 5
First three distinct visitors:
Harsh
Rahul
Amit
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12>
```

Learning Outcome: Learned to eliminate duplicate entries, count unique users, and preserve their first-seen order.

Problem 3: Leaderboard (Advanced)

Aim: To create a player leaderboard using Map and sort players by score, breaking ties alphabetically by name.

Code Screenshot:



```java
import java.util.*;
public class Leaderboard {
    Run | Debug
    public static void main(String[] args) {
        Map<String, Integer> scores = new HashMap<>();

        scores.put("Harsh", 95);
        scores.put("Rahul", 88);
        scores.put("Amit", 95);
        scores.put("Priya", 92);
        scores.put("Neha", 88);
        int topN = 3;
        List<Map.Entry<String, Integer>> players =
                new ArrayList<>(scores.entrySet());

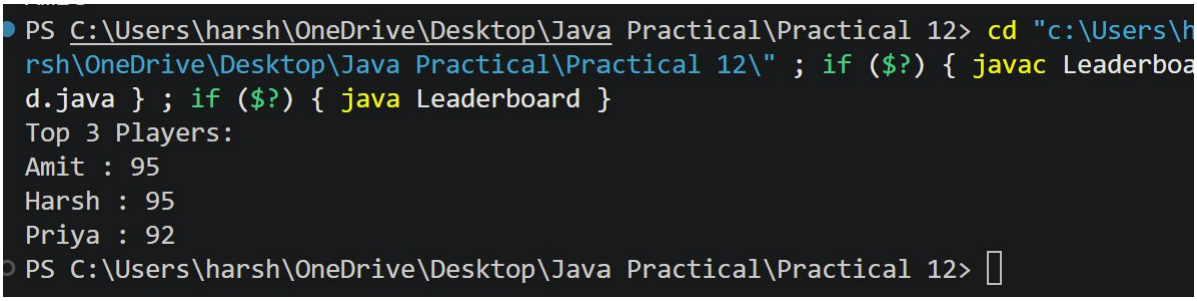
        players.sort(
                Comparator.comparingInt(
                        (Map.Entry<String, Integer> e) -> e.getValue()
                ).reversed()
                .thenComparing(Map.Entry::getKey)
        );

        System.out.println("Top " + topN + " Players:");

        for (int i = 0; i < Math.min(topN, players.size()); i++) {
            Map.Entry<String, Integer> player = players.get(i);

            System.out.println(
                    player.getKey() + " : " + player.getValue());
        }
    }
}
```

Output Screenshot:



```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12\" ; if ($?) { javac Leaderboard.java } ; if ($?) { java Leaderboard }
Top 3 Players:
Amit : 95
Harsh : 95
Priya : 92
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 12>
```

Learning Outcome: Learned to sort map entries using Comparator and display the top-N players according to their scores.

PRACTICAL - 13

Problem 1: Generic Box, Pair and First Element

Aim: To implement a generic Box<T>, a Pair<A, B>, and a generic method that returns the first element of an array.

Code Screenshot:

```java
class Box<T> {
    private T value;
    public void set(T value) {
        this.value = value;}
    public T get() {
        return value;}}
class Pair<A, B> {
    private A first;
    private B second;
    public Pair(A first, B second) {
        this.first = first;
        this.second = second;}
 public A getFirst(){
        return first;}
    public B getSecond() {
        return second;}}
public class GenericDemo {
    public static <T> T firstElement(T[] array) {
        if (array == null || array.length == 0) {
            throw new IllegalArgumentException("Array is empty");
        }
        return array[0];}
    Run | Debug
    public static void main(String[] args) {
        Box<Integer> box1 = new Box<>();
        box1.set(value: 100);
        System.out.println("Box value: " + box1.get());
        Box<String> box2 = new Box<>();
        box2.set(value: "Hello Java");
        System.out.println("Box value: " + box2.get());
        Pair<String, Integer> pair =
                new Pair<>(first: "Harsh", second: 20);
        System.out.println("Pair: "
                + pair.getFirst() + ", " + pair.getSecond());
        Integer[] numbers = {10, 20, 30};
        String[] names = {"Harsh", "Rahul"};
        System.out.println(
                "First number: " + firstElement(numbers));
        System.out.println(
                "First name: " + firstElement(names));
    }
}
```

Output Screenshot:

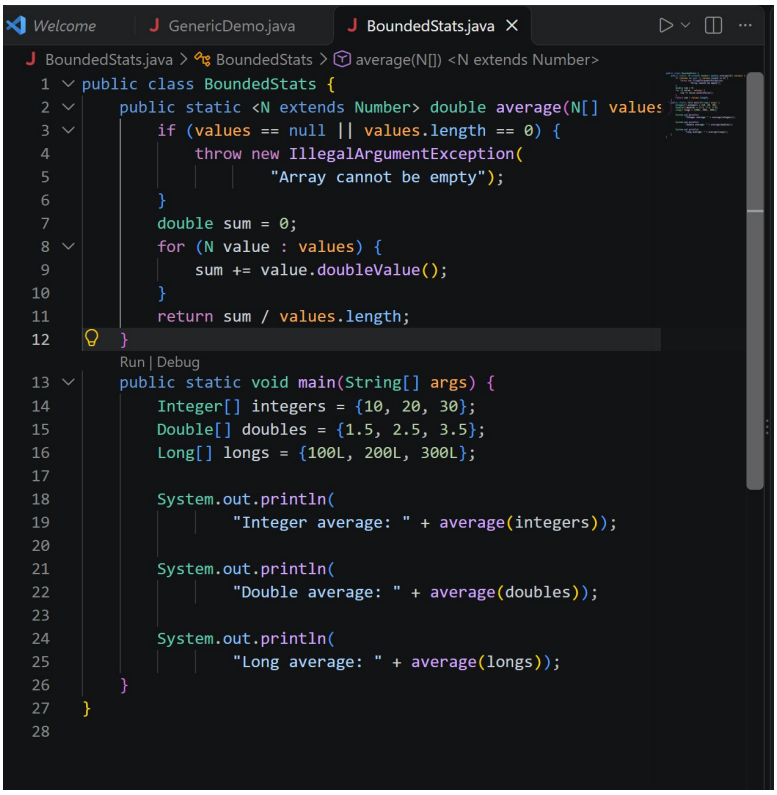
```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 13> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 13\" ; if ($?) { javac GenericDemo.java } ; if ($?) { java GenericDemo }
Box value: 100
Box value: Hello Java
Pair: Harsh, 20
First number: 10
First name: Harsh
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 13>
```

Learning Outcome: Learned to develop reusable, type-safe generic classes and methods for different data types.

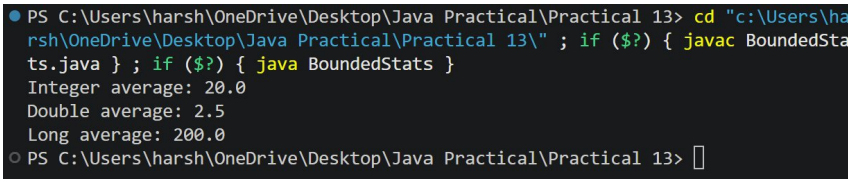
Problem 2: Bounded Statistics

Aim: To implement a generic average() method using the bounded type parameter N extends Number for Integer, Double, and Long arrays.

Code Screenshot:



Output Screenshot:



Learning Outcome: Learned to use bounded generics to perform numerical operations on different Number types while ensuring type safety.

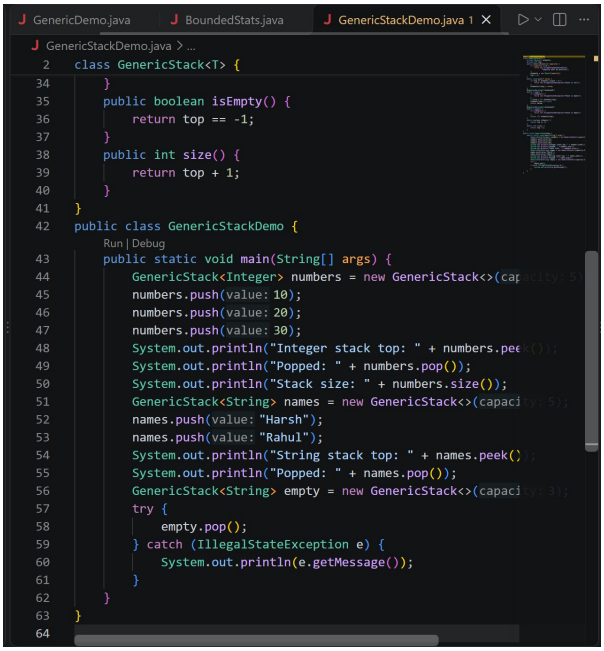
Problem 3: Generic Stack (Advanced)

Aim: To implement a generic stack using an array with push(), pop(), and peek() operations.

Code Screenshot:



```java
import java.util.Arrays;
class GenericStack<T> {
    private Object[] elements;
    private int top;
    public GenericStack(int capacity) {
        if (capacity <= 0) {
            throw new IllegalArgumentException(
                    "Capacity must be positive");
        }
        elements = new Object[capacity];
        top = -1;
    }
    public void push(T value) {
        if (top == elements.length - 1) {
            throw new IllegalStateException("Stack is full");
        }
        elements[++top] = value;
    }
    @SuppressWarnings("unchecked")
    public T pop() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }
        T value = (T) elements[top];
        elements[top--] = null;
        return value;
    }
    @SuppressWarnings("unchecked")
    public T peek() {
        if (isEmpty()) {
            throw new IllegalStateException("Stack is empty");
        }
        return (T) elements[top];
```



```java
class GenericStack<T> {
    }
    public boolean isEmpty() {
        return top == -1;
    }
    public int size() {
        return top + 1;
    }
}
public class GenericStackDemo {
    Run | Debug
    public static void main(String[] args) {
        GenericStack<Integer> numbers = new GenericStack<>(cap
        numbers.push(value: 10);
        numbers.push(value: 20);
        numbers.push(value: 30);
        System.out.println("Integer stack top: " + numbers.pee
        System.out.println("Popped: " + numbers.pop());
        System.out.println("Stack size: " + numbers.size());
        GenericStack<String> names = new GenericStack<>(capaci
        names.push(value: "Harsh");
        names.push(value: "Rahul");
        System.out.println("String stack top: " + names.peek()
        System.out.println("Popped: " + names.pop());
        GenericStack<String> empty = new GenericStack<>(capaci
        try {
            empty.pop();
        } catch (IllegalStateException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

Output Screenshot:

```
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 13> cd "c:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 13\" ; if ($?) { javac GenericStackDemo.java } ; if ($?) { java GenericStackDemo }
Integer stack top: 30
Popped: 30
Stack size: 2
String stack top: Rahul
Popped: Rahul
Stack is empty
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 13>
```

Learning Outcome: Learned to create a reusable stack for different data types and handle stack overflow and underflow conditions.

PRACTICAL – 14

Problem 1: Layered Application and JUnit Testing

Aim: To develop a layered Java application consisting of a model, a service using ArrayList, and a menu-driven main program, and test its functionality using JUnit.

Code Screenshot:

Counter.java

```java
public class Counter {
    private int count;

    public int getCount() {
        return count;
    }

    public void increment() {
        count++;
    }
}
```

CounterService.java

```java
import java.util.ArrayList;

public class CounterService {
    private final Counter counter = new Counter();
    private final ArrayList<String> history = new ArrayList<>(

    public void add() {
        counter.increment();
        history.add("Added: " + counter.getCount());
    }

    public int getCount() {
        return counter.getCount();
    }

    public ArrayList<String> getHistory() {
        return new ArrayList<>(history);
    }

    public void performAction(int action) {
        if (action == 1) {
            add();
        } else if (action != 0) {
            throw new IllegalArgumentException("Invalid actior
        }
    }
}
```

```java
import java.util.Scanner;
public class CounterApp {
    Run | Debug
    public static void main(String[] args) {
        CounterService service = new CounterService();
        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println("\n1. Add");
                System.out.println("2. Show Count");
                System.out.println("3. Show History");
                System.out.println("0. Exit");
                System.out.print("Enter choice: ");
                if (!sc.hasNextInt()) {
                    System.out.println("Enter a valid number.");
                    sc.next();
                    continue;
                }
                int choice = sc.nextInt();
                try {
                    switch (choice) {
                        case 1:
                            service.performAction(1);
                            System.out.println("Count increase
                            break;
                        case 2:
                            System.out.println("Count: "
                                    + service.getCount());
                            break;
                        case 3:
                            System.out.println(service.getHist
                            break;
                        case 0:
                            System.out.println("Goodbye!");
                            case 0:
                                System.out.println("Goodbye!");
                                return;
                            default:
                                service.performAction(choice);
                        }
                    } catch (IllegalArgumentException e) {
                        System.out.println(e.getMessage());
                    }
                }
            }
        }
    }
```

```java
import org.junit.jupiter.api.Test;
import static org.junit.jupiter.api.Assertions.*;
public class CounterServiceTest {

    @Test
    void addingIncreasesCount() {
        CounterService service = new CounterService();

        service.add();

        assertEquals(1, service.getCount());
    }

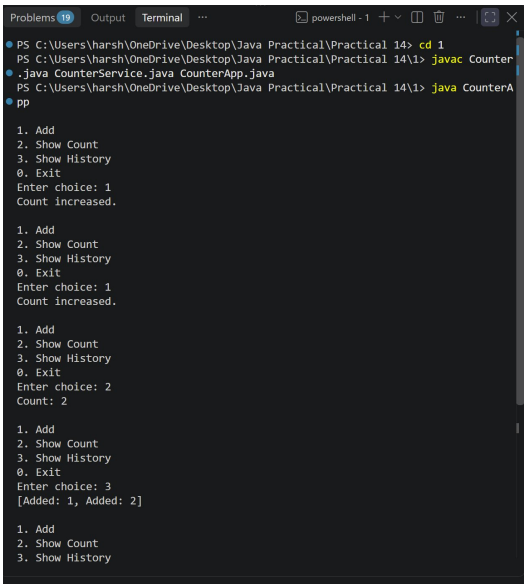
    @Test
    void invalidActionThrowsException() {
        CounterService service = new CounterService();

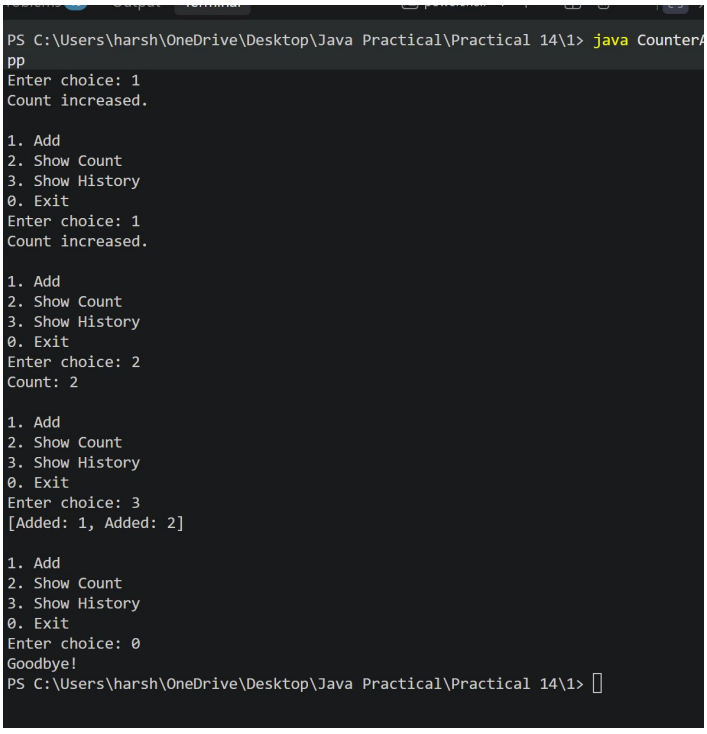
        assertThrows(
                IllegalArgumentException.class,
                () -> service.performAction(99)
        );
    }

    @Test
    void newServiceStartsAtZero() {
        CounterService service = new CounterService();

        assertEquals(0, service.getCount());
    }
}
```

Output Screenshot:





Learning Outcome: Learned to organize an application into separate layers and write JUnit tests to verify functionality and exception handling.

Problem 2: Packages, Runnable JAR and Save/Load Testing (Advanced)

Aim: To organize a Java application into packages, build a runnable JAR, and test whether saved data can be reloaded without changes.

Code Screenshot:



Main.java

src > main > java > com > example > Main.java > { } com.example

```java
package com.example;

import com.example.service.CounterService;
import java.nio.file.Files;
import java.nio.file.Path;
import java.util.Scanner;

public class Main {
    Run | Debug
    public static void main(String[] args) throws Exception {
        CounterService service = new CounterService();
        Path file = Path.of(first: "counter-data.txt");

        if (Files.exists(file)) {
            service.load(file);
            System.out.println("Loaded count: " + service.getCount());
        }

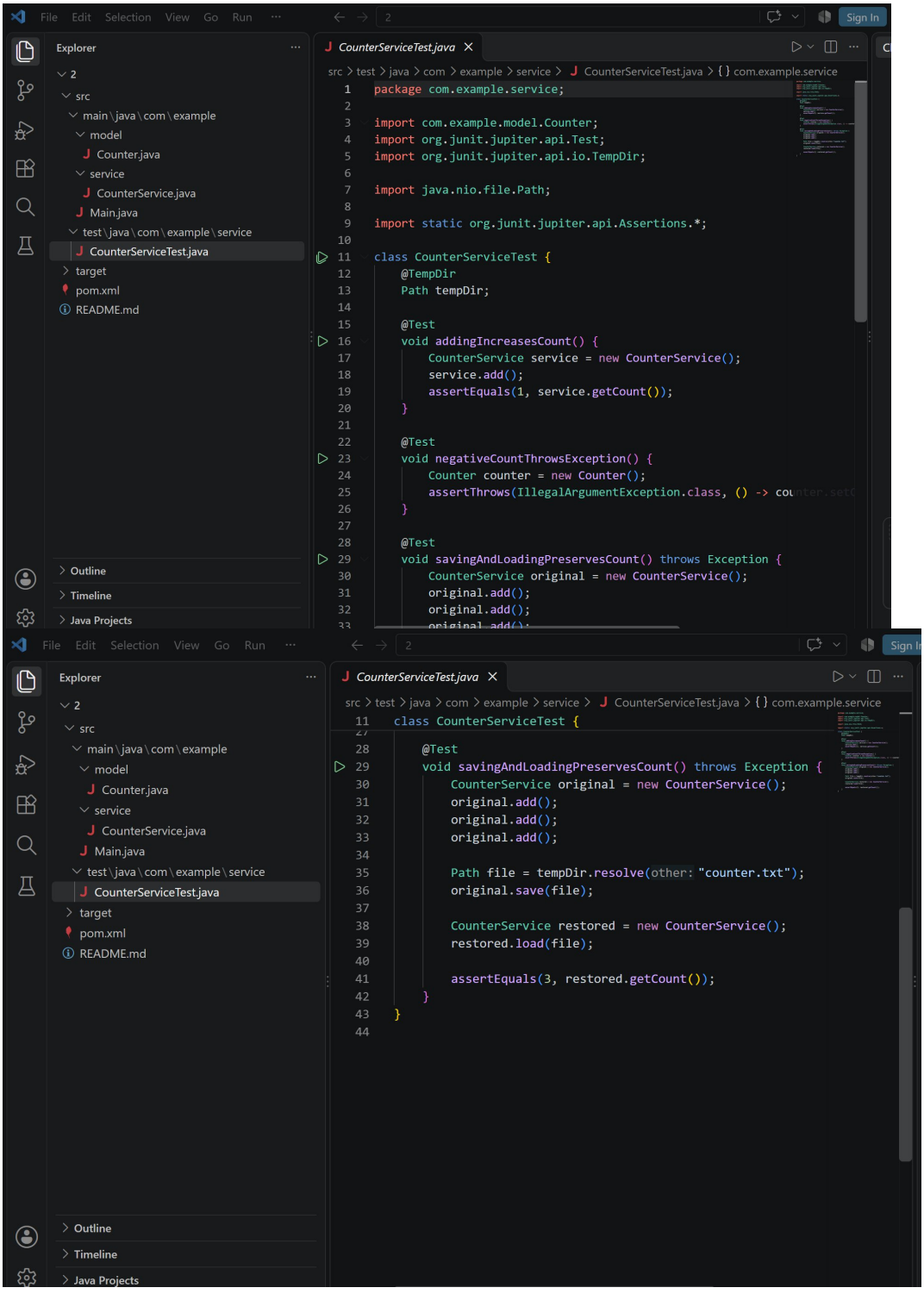
        try (Scanner sc = new Scanner(System.in)) {
            while (true) {
                System.out.println(x: "\n1. Add");
                System.out.println(x: "2. Show Count");
                System.out.println(x: "3. Save");
                System.out.println(x: "4. Load");
                System.out.println(x: "0. Exit");
                System.out.print(s: "Choice: ");

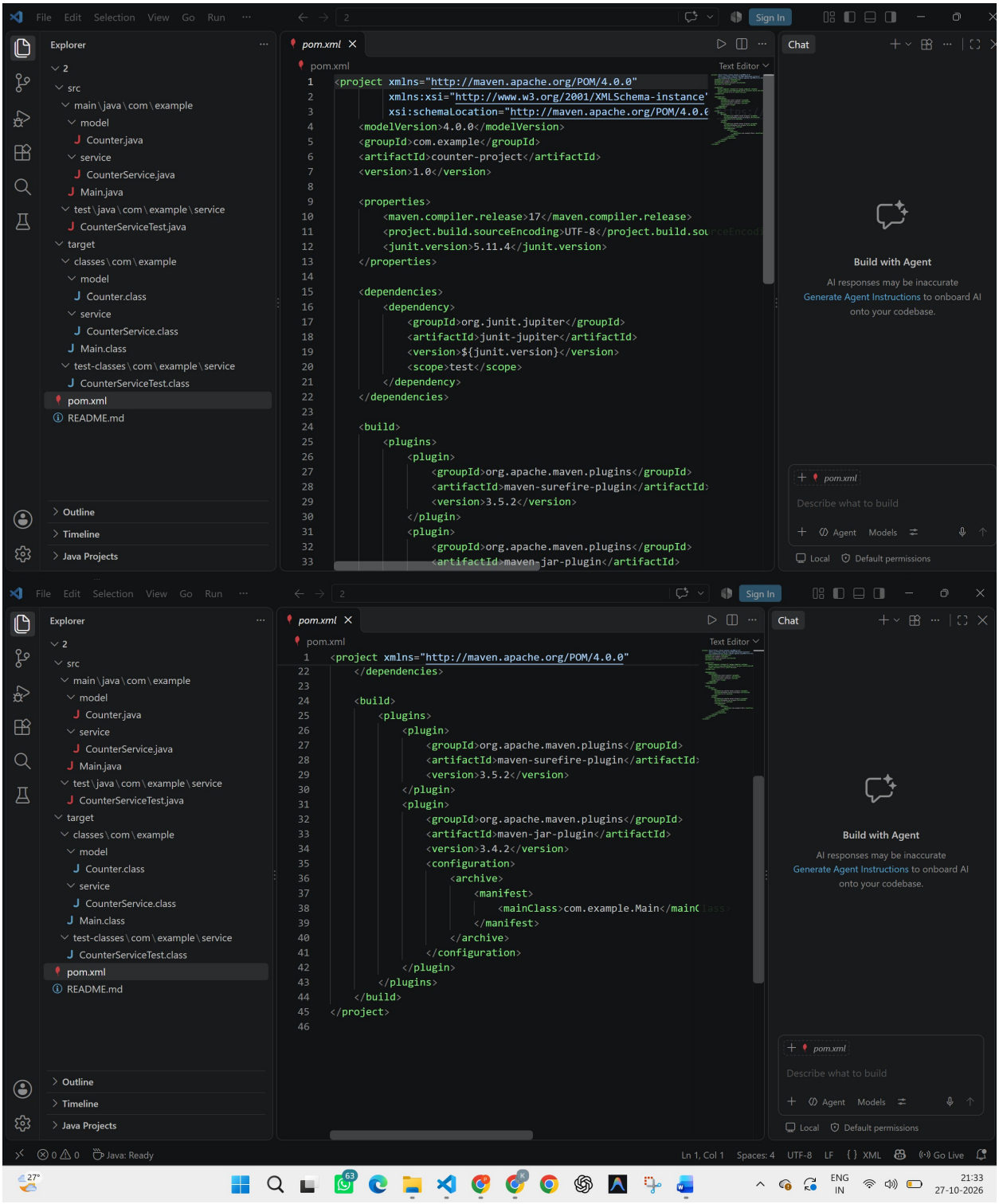
                if (!sc.hasNextInt()) {
                    System.out.println(x: "Invalid input.");
                    sc.next();
                    continue;
                }
```

Main.java

src > main > java > com > example > Main.java > { } com.example

```java
public class Main {
    public static void main(String[] args) throws Exception {
                try {
                    switch (choice) {
                        case 1:
                            service.add();
                            System.out.println(x: "Count increased.");
                            break;
                        case 2:
                            System.out.println("Count: " + service.getCount());
                            break;
                        case 3:
                            service.save(file);
                            System.out.println(x: "Data saved.");
                            break;
                        case 4:
                            service.load(file);
                            System.out.println(x: "Data loaded.");
                            break;
                        case 0:
                            System.out.println(x: "Goodbye!");
                            return;
                        default:
                            System.out.println(x: "Invalid action.");
                    }
                } catch (IllegalArgumentException e) {
                    System.out.println(e.getMessage());
                }
            }
        }
    }
}
```


File Edit Selection View Go Run
Sign In
Explorer
CounterServiceTest.java
src > test > java > com > example > service > CounterServiceTest.java > { } com.example.service
2
src
main\java\com\example
model
Counter.java
service
CounterService.java
Main.java
test\java\com\example\service
CounterServiceTest.java
target
pom.xml
README.md
Outline
Timeline
Java Projects
1 package com.example.service;
3 import com.example.model.Counter;
4 import org.junit.jupiter.api.Test;
5 import org.junit.jupiter.api.io.TempDir;
7 import java.nio.file.Path;
9 import static org.junit.jupiter.api.Assertions.*;
11 class CounterServiceTest {
12 @TempDir
13 Path tempDir;
15 @Test
16 void addingIncreasesCount() {
17 CounterService service = new CounterService();
18 service.add();
19 assertEquals(1, service.getCount());
20 }
22 @Test
23 void negativeCountThrowsException() {
24 Counter counter = new Counter();
25 assertThrows(IllegalArgumentException.class, () -> co
26 }
28 @Test
29 void savingAndLoadingPreservesCount() throws Exception {
30 CounterService original = new CounterService();
31 original.add();
32 original.add();
11 class CounterServiceTest {
28 @Test
29 void savingAndLoadingPreservesCount() throws Exception {
30 CounterService original = new CounterService();
31 original.add();
32 original.add();
33 original.add();
35 Path file = tempDir.resolve(other: "counter.txt");
36 original.save(file);
38 CounterService restored = new CounterService();
39 restored.load(file);
41 assertEquals(3, restored.getCount());
42 }
43 }


File Edit Selection View Go Run
Explorer
pom.xml
Chat
Build with Agent
AI responses may be inaccurate
Generate Agent Instructions to onboard AI onto your codebase.
Describe what to build
Agent Models
Local Default permissions
Outline
Timeline
Java Projects
Java: Ready
Ln 1, Col 1 Spaces: 4 UTF-8 LF XML Go Live
ENG IN
21:33
27-10-2026

Output Screenshot:

```
Problems  Output  Terminal  ...                                  powershell

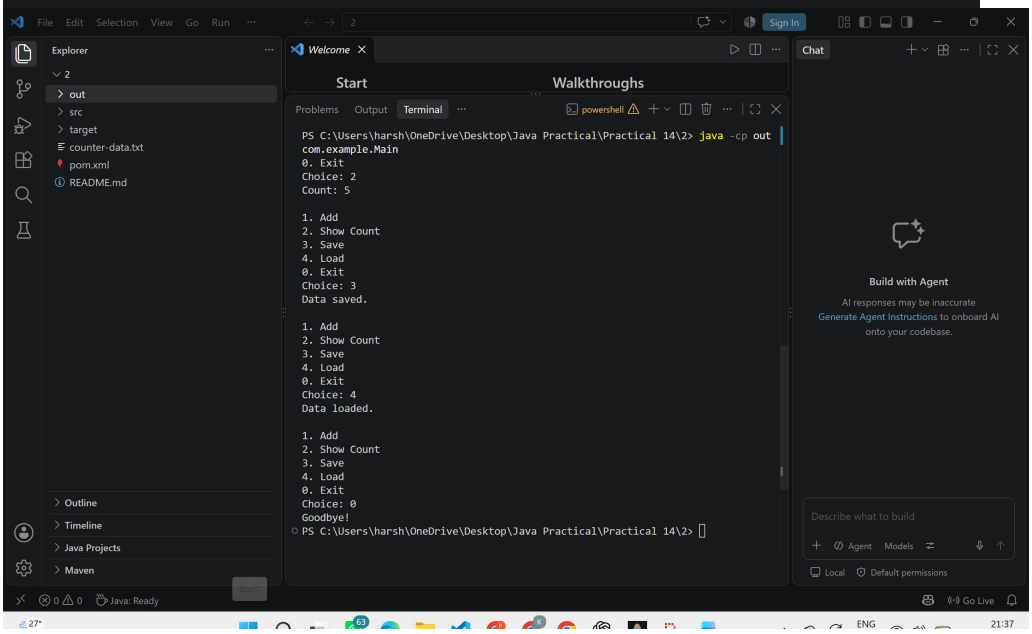
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 14\2> javac -d out
src\main\java\com\example\model\Counter.java src\main\java\com\example\service\
CounterService.java src\main\java\com\example\Main.java
PS C:\Users\harsh\OneDrive\Desktop\Java Practical\Practical 14\2> java -cp out
com.example.Main

1. Add
2. Show Count
3. Save
4. Load
0. Exit
Choice: 1
Count increased.

1. Add
2. Show Count
3. Save
4. Load
0. Exit
Choice: 1
Count increased.

1. Add
2. Show Count
3. Save
4. Load
0. Exit
Choice: 1
Count increased.

1. Add
2. Show Count
```



Learning Outcome: Learned to package a Java application, execute it as a runnable JAR, and verify data persistence using automated tests.